Delphi Project Handoff Engineering Guide

Architecture, data contracts, validation and maintenance

Experimental release 0.8.0.3 | Guide edition 52

Last changed 16 October 2026

From architecture and data ownership to source analysis, storage, testing and release

Experimental software

This manual describes Delphi Project Handoff. Information about a selected Delphi application is presented separately by the program.

Contents

PART 1: Product contracts and architecture....3
1. Engineering purpose and reading map....3
2. System boundaries and design rules....4
3. Layer and responsibility map....5
4. Startup and application lifecycle....7
5. Background request and response protocol....8
6. End-to-end flows an engineer must preserve....9
PART 2: Source acquisition, syntax, and semantics....10
7. Source bytes, ranges, and safe acquisition....10
8. Project inventory and reading configuration....11
9. Lexer, preprocessing, and intermediate representation....12
10. Pascal, forms, and resource parsing....13
11. Symbols, resolution, control flow, and focused analyses....14
PART 3: Persistence, authored knowledge, and native presentation....15
12. SQLite store ownership, migrations, and transactions....15
13. Generations, source identity, and cached views....16
14. Authored revisions, links, and review state....17
15. Structured tasks, teaching cards, and personal progress....18
16. Pictures, artifacts, and anchored callouts....20
17. Learner projections and native binding....21
PART 4: Publication, search, and operational safety....22
18. Publication model and export contract....22
19. Search, browser navigation, and supporting integrations....24
PART 5: Building, testing, diagnosis, and release....25
20. Build environment and source layout....25
21. Meaningful test selection and execution....26
22. Package, deploy, and verify extracted copies....27
23. Diagnose failures at their owning layer....29
24. Maintain documentation as an engineering artifact....30
PART 6: Technical reference....31
25. Source units and public contracts....31
26. Persisted schema and migration reference....50
27. Engineering change and release checklists....57

PART 1: Product contracts and architecture

1. Engineering purpose and reading map

1.1. The system being maintained

Delphi Project Handoff, abbreviated DPH, is a Windows desktop application implemented with Delphi and FireMonkey. Its product responsibility is to prepare an evidence-based maintenance handoff for one unfamiliar Delphi application. It reads project, source and form material; persists captured facts and attributed knowledge; presents a handoff dossier and specialist views; and publishes a portable reading edition. The target application's source is input to this process, not a workspace that DPH rewrites or executes.

This manual explains the implementation, ownership boundaries, persisted contracts, failure behavior and maintenance procedures. The User Guide explains how to operate the visible product. Read the architecture and end-to-end flows before modifying an analyzer or save path. Use the source-unit and schema references at the end to find exact declarations and constraints, then consult the named implementation method for its complete behavior.

The architecture contains more technical mechanisms than the learner interface exposes. Source identity, immutable generations, database ownership and publication integrity support reliable instruction. They must not become the organizing subject of the beginner route. Conversely, an engineer needs to understand them deeply to prevent stale or unreviewed content being presented as established knowledge.

The visible workflow has six tabs: Getting started, Handoff dossier, Learning library, Project map, Screenshots, and Application diagrams. Open application starts a temporary source capture; closing or changing projects offers Save, Discard, and Cancel before a permanent .dphdb is created. Open saved project restores a .dphdb and its adjacent .views folder. DPH.Handoff.Dossier renders the four dossier sections from the selected publication model; the renderer does not execute source or turn an inferred relationship into an observed result. Form-preview generation is deferred until Screenshots is opened to reduce the cost of restoring a saved project. Older authoring, lesson, and specialist controls remain serialized or implemented for compatibility, but must not be described as visible first-use commands.

1.2. Three independent kinds of truth

Captured source facts describe what DPH established from a particular source set and configuration. Human knowledge describes what a named contributor supplied. Observations describe separately imported material claimed to represent runtime evidence. These are distinct provenance channels. Parser success cannot verify a business explanation, a contribution cannot turn an unresolved call into a statically confirmed target, and a supplied observation does not mean DPH executed the program.

Preserve these distinctions in database columns, JSON projections, UI wording and exports. A current generation may still have incomplete semantic understanding. A reviewed explanation can become stale when its linked source changes. An internally consistent export may contain intentionally historical or partial material when explicitly permitted. A single green success flag cannot replace those separate states.

1.3. How to use the technical references

Chapters 2 through 6 establish boundaries and orchestration. Chapters 7 through 11 explain acquisition, parsing and semantic evidence. Chapters 12 through 16 cover persistence, knowledge, progress and images. Chapters 17 through 19 describe projection, publication and browser policy. Chapters 20 through 24 cover building, testing, deployment and diagnosis.

The source reference is generated from the current project units and distinguishes public declarations from explanatory narrative. The schema reference is derived from numbered SQL migrations, retaining actual table columns and constraints. Neither reference should be mistaken for an invitation to manipulate a production store manually; storage mutations must continue through the writer and lifecycle contracts.

2. System boundaries and design rules

2.1. Inputs outputs and side effects

Inputs include the selected .dproj, .dpr or .groupproj entry, reachable allowed source files, form/resource data, source-reading configuration, authored explanations and imported PNG or observation files. Source acquisition is bounded to the selected root and explicit approved dependency roots. Paths and reparse behavior are checked so that an apparently local reference does not silently acquire unrelated filesystem content.

Outputs include the .dphdb knowledge store, adjacent managed .views content, local workspace preferences, generated learning pages and fresh offline publication folders. DPH's own build produces executables and intermediate units under bin and dcu. Publication destinations and the knowledge database must remain outside the selected target source tree.

The selected target is not compiled or run during ordinary analysis. DPH's own test programs and developer fixtures may be executed as explicit development validation. Keep those operations separate from claims about an arbitrary target application's runtime behavior.

2.2. Native forms are serialized assets

DPH.UI.Main.fmx and the dedicated dialog .fmx files define ordinary controls, labels, dimensions, alignment, scrolling and event hookups. Engineers should be able to open them in RAD Studio and modify ordinary layout properties in the Object Inspector. Do not replace these assets with an unreviewable runtime-only UI construction path.

Generated HTML is a view inside serialized browser controls. Graph and image overlays use the established rendering paths; their style prototypes remain serialized where implemented. Runtime response handling binds content and state to those controls. Layout changes require both a resource-instantiation check and visual inspection at maximized and restored sizes.

2.3. Local storage is not a target business database

The SQLite store is DPH's internal knowledge database. Modules describing database engines, dictionaries, schema, bindings or rules analyze or capture explanations about a selected application; they are not general business database administration engines. Their existence in source does not mean every specialist page belongs in the learner selector.

The ordinary preparation selector is intentionally limited. Legacy serialized pages and backend capabilities remain in the repository and may support existing records or deeper inspection. An engineer must not advertise a hidden page as an accessible current workflow or add a menu entry solely because a class exists.

2.4. Scope and evidence limits

The historical requirements ledger preserves original implementation assessments. Learning_Requirements_Reconciliation maps every original record to the revised teaching contract, including superseded technical obligations. A scope decision is not an implementation of the original specialist requirement. Preserve that distinction in engineering status and release claims.

Teach only what the captured source and attributed contributions support. A static call graph is not a runtime trace, a reconstructed form is not a screenshot, and absence from a bounded result list is not proof of absence from the whole program. Qualify uncertain dispatch, dynamic controls, external services and unwritten business intent explicitly.

DPH.Learning.Discovery scans eligible documents only within the selected project directory. It bounds traversal and samples documents and logs before building Learning library entries. The text-quality filter must count Unicode letters, not only ASCII Latin letters; otherwise readable Arabic, CJK, Cyrillic, and other guides disappear even though DPH's interface has offline language packs. A discovered file with no usable text should carry a visible reason. PDF and older word-processing extraction depends on LibreOffice at the standard Windows path; DOCX and supported text formats are processed directly. Referenced public HTTPS pages may be contacted only when Read public websites is enabled. Discovery rejects private IPv4 resolutions and uses WinHTTP's request-level hostname-resolution override to pin the connection to the checked public IPv4 address while retaining the original host for TLS verification. It uses no proxy, disables redirects, cookies, and authentication, and caps responses at 128 KiB. If the Windows 10 21H1-or-later override is unavailable, the request fails closed. Network isolation still needs controlled adversarial testing before a production release.

3. Layer and responsibility map

3.1. Acquisition and syntax layers

DPH.Project.Source acquires bytes and maps decoded character positions back to byte ranges. DPH.Project.Inventory discovers allowed project members, reading configuration, search paths and dependencies. DPH.Parse.Lexer produces tokens. DPH.Parse.Preprocessor attaches conditional state and resolves permitted includes. DPH.Parse.Pascal, Forms and Resources produce structured syntax or qualified opaque evidence.

DPH.Project.Analysis coordinates parse results into a TDPHScan and provides staging operations. This layer owns source and parse-result lifetimes. The semantic layer consumes those results rather than rereading arbitrary files or running the compiler to infer behavior.

3.2. Semantic and projection layers

DPH.Analysis.Model defines symbols, relationships, flow nodes and edges. DPH.Analysis.Symbols binds source constructs, resolves supported references and caches routine summaries. DPH.Analysis.Flow builds control flow. Async, SQL, Persistence and Quality add focused analyses with their own qualifications.

DPH.Workspace.Model captures a reusable JSON view and provides object lookup, search, coverage, questions and structured teaching-record validation. Specialized Workspace units project graphs, screen designs, layout rules, dependencies and comparisons. DPH.Analysis.Explanations turns captured facts and selected contributions into beginner-oriented lessons.

3.3. Storage orchestration and presentation

DPH.Storage.Database owns SQLite, migrations, writer ownership, immutable generation facts, knowledge revisions, search indexes and managed artifacts. DPH.Storage.Backup preserves a consistent store with its managed files. DPH.Workspace.Worker serializes UI requests on a background thread and returns owned response objects through bounded queues.

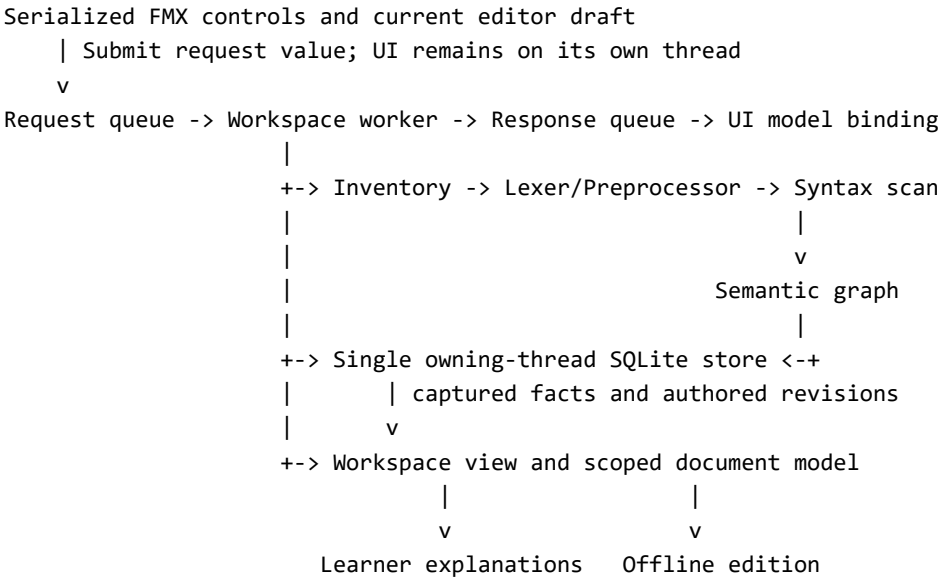
DPH.UI.Main binds requests, response state and native controls. Dedicated dialogs collect structured profile, glossary, action-purpose, dependency and dictionary material. DPH.Publish.Model builds the scoped publication contract. DPH.Publish.Export writes and validates editions. DPH.Publish.BrowserPolicy gates embedded navigation. DPH.Publish.Translation and the pinned translation dependency are supporting integrations, not an assumption that every learner page performs remote translation.

3.4. A responsibility test for a proposed change

If a field is displayed incorrectly, inspect the form and binding first. If a typed teaching record is rejected, inspect its model validator and SaveKnowledge canonical-body checks. If a call target is wrong, inspect symbol binding and semantic caching rather than adding a UI exception. If an export exposes excluded content, inspect the publication model before the HTML renderer.

Each correction should live at the layer that owns its contract. Fixing a symptom in a later layer can conceal inconsistent underlying facts and produce different behavior between the native reader and exported lessons.

3.5. Read the data and ownership path



This is an architectural reading map rather than a runtime trace. The worker constructs and owns storage operations; the UI adopts owned response data and binds controls. The native lesson and offline edition share explanatory/model contracts, while export additionally applies its scope and privacy boundary.

4. Startup and application lifecycle

4.1. Program entry and resource instantiation

DelphiProjectHandoff.dpr initializes the FireMonkey application and creates the main form. The .dproj supplies build configuration and project dependencies. Main-form creation initializes collections, editor state, workspace catalog, selectors, worker orchestration and the introductory view. Form resources must instantiate without missing event methods or mismatched component fields.

The baseline test creates the serialized form and checks component bindings. It catches failures that a compiler can miss when a .fmx event name refers to an absent method or when a field declaration no longer agrees with a serialized component. Treat that test as necessary for form changes, while recognizing that it does not verify visual usability.

4.2. Locating bundled assets

The main form determines its base directory from the executable location and searches upward for the migrations directory within the implemented bounded search. That base locates migrations, help and samples. This supports both a development executable under bin and a deployed executable beside its content folders.

A desktop package must preserve that structure. A successful source-tree run does not prove a package contains the same files. Extract the binary ZIP elsewhere and test migration discovery, local help, current guide availability and browser-loader availability from the extracted location.

4.3. Preferences and local catalog

TDPHWorkspaceCatalog stores known database paths and learning_progress in the existing per-user workspace.json location under the user's local application data. It migrates the established single database entry into a catalog without treating catalog removal as deletion of the store. The catalog is separate from a .dphdb and therefore is not automatically transferred with that database.

Catalog construction validates its version and structure. Remember, Forget and Save update local navigation state. An absent database remains representable as a missing catalog entry. This lets the user locate moved files without implying that source or knowledge has been deleted.

4.4. Closing and cancellation boundaries

FormCloseQuery protects pending editor work. FormDestroy stops the worker, waits for its termination and releases owned models and collections. Do not free controls or shared state while a response can still be applied. Worker shutdown closes queues and drains response objects through the established ownership path.

A user-requested close is not a transaction rollback API for arbitrary already-committed work. Preserve storage atomicity so interruption cannot publish half a generation or partially update a knowledge revision. On reopening, interrupted generation recovery marks incomplete work rather than presenting it as current.

5. Background request and response protocol

5.1. Request structure

TDPHWorkspaceRequest is a value record containing command, database, source entry, configuration, platform, define/root arrays, generation and revision identifiers, authored text and links, export options, and screenshot/history parameters. Submit initializes a default record and fills the fields needed by that command.

The worker uses a TThreadedQueue of eight requests and a separate queue of eight response objects. Submit raises an explicit queue-full error when PushItem cannot accept the record. UI busy state prevents ordinary overlapping user commands; it is not a replacement for storage ownership or revision checks.

5.2. Worker thread lifetime and COM

TDPHWorkspaceWorker is a non-free-on-terminate TThread. Execute initializes COM for the worker's apartment, constructs analyzer instances, consumes requests and releases resources on exit. Store instances are created and used on the owning worker thread. FireDAC connections and writer-thread checks must not be bypassed by calling the same store from the UI thread.

Requests for opening, scanning, saving, reviewing, searching, publishing or importing artifacts share this orchestration. Some commands return specialized response data without replacing the complete workspace view. Do not treat every response as a full analysis completion.

5.3. Response ownership and stale result guards

TDPHWorkspaceResponse owns its Document, Knowledge, Screenshots and other JSON result fields. Its destructor frees these fields. When the UI adopts a field, it transfers ownership by assigning it to the retained model and setting the response's field to nil. Omitting that nil assignment can double-free an adopted object; cloning without a reason adds unnecessary memory and obscures ownership.

Generation-sensitive search, comparison and review responses are accepted only when their recorded generation agrees with the displayed view where guarded. Screenshot-history responses also carry tokens and database identity. These checks prevent a delayed result for an older selection from replacing the current workspace.

5.4. Draft state across asynchronous save

The UI records which structured task or teaching card is being saved. On success it reloads knowledge, finds that stable identity and returns to the reader. Failed saves clear the in-flight identity as appropriate and preserve editable draft content. The editor must not declare itself saved simply because a background response arrived.

For general knowledge, the saved snapshot includes title, body, document destination, audience, links, question metadata and structured JSON. FKnowledgeEdited is cleared only when the current editor still matches the submitted snapshot. This protects an edit made after submission from being accidentally marked clean by an earlier save response.

6. End-to-end flows an engineer must preserve

6.1. First source attachment

The UI obtains a project entry and an outside-source database destination. Inventory validates the selected project and source-reading context. The worker rejects DPH's own project before scanning. The store opens or initializes its singleton project and selected configuration, while source files are observed into a manifest.

The analyzer parses supported inputs, the semantic analyzer builds the graph, facts are staged in a generation and source is revalidated before publication. A workspace view is saved and returned. The native Handoff dossier is then rendered from that captured view and current knowledge. A failure before publication must leave the earlier published head available.

6.2. Opening cached lessons

Open uses the database's recorded project root and stored view instead of pretending to acquire missing source. The UI can present a detached captured view and saved knowledge. Detached status must remain explicit, particularly for review and publication decisions that require current source.

Opening a cache is not equivalent to a full scan. A normal launch opens Getting started without loading the last database; only an explicit selection or .dphdb command-line argument opens a saved capture. If the recorded source entry is still available under the recorded root, the worker verifies its manifest, stale flag, and analyzer/discovery versions in the background; a stale flag, changed fingerprint, or changed version triggers an incremental scan. If unchanged, it reports verification without scanning again. Opening the source entry with Open application uses the same incremental comparison. It should not perform every expensive optional report merely because the form has serialized tabs for them. Build deeper projections on demand for their active destination.

6.2.1. What the dossier STALE label means

RefreshLearning marks the document model stale when its source is detached. An attached capture with analysis_complete false is PARTIAL, not STALE; analysis_complete is true only when both syntax and semantic passes complete. DPH.Handoff.Dossier renders Static inspection only for every capture because it never executes the selected application. The store separately tracks analysis_stale for source changes, interrupted generations and reassessment. On an explicit saved-project open with source available, the worker now includes that flag in its background rescan decision; a successful current generation clears it. Unsupported syntax, unresolved connections, or missing material may still leave a refreshed capture PARTIAL. A detached saved capture remains marked STALE until its source can be verified.

6.3. Saving and reviewing knowledge

An editor produces text, metadata and the expected current revision. The worker validates and sends the save to TDPHKnowledgeStore. The store checks record type, canonical body, project-scoped links and optimistic revision identity, then appends a revision and updates the appropriate head. The returned knowledge projection drives the reader.

Review is another command with its own reviewer, revision, generation and current-source checks. A save does not transition automatically to a verified review. If source changed during checking, refuse to present that review as current until reassessment establishes the correct evidence context.

6.4. Publishing a learning edition

The worker builds TDPHDocumentModel from the selected workspace view, saved knowledge, images, audience, seeds and exclusions. Scoping and redaction occur before rendering. Export writes a fresh destination, includes indexes and provenance, computes checksums and validates the resulting edition.

The portable edition now includes handoff-0.html through handoff-3.html from the same scoped, redacted document model as the older learning.html. The native export preview still opens the older learning page, so its default preview flow remains to be aligned with the dossier. A consistent technical index does not establish human instructional quality. Test the handoff, a supplied task, a picture and an object explanation in the independent export as well as the native reader.

PART 2: Source acquisition, syntax, and semantics

7. Source bytes, ranges, and safe acquisition

7.1. TDPHSource is the byte to text boundary

TDPHSource retains source path, original bytes, decoded text, encoding identity, line-ending information and a character-to-byte offset array. The acquisition method opens and reads the file under its path checks, rejects source larger than the implemented 16 MiB limit and detects a size change during acquisition.

Original byte hashes establish source identity. Do not hash normalized text and then label that result as the original file hash. Line-ending conversion, a byte-order mark and different encodings can yield equal-looking text with different original bytes. Evidence ranges refer back to the acquired bytes.

7.2. Encoding precedence

The reader recognizes UTF-8 BOM, UTF-16 little endian and UTF-16 big endian before considering the explicit code page. With no byte-order mark and no selected legacy code page, it attempts the implemented UTF-8 decoding path. An explicit Windows code page supports older source where authorized. Read the actual decoder exception and fallback branch before changing encoding behavior.

This is source decoding, not translation. UI explanations should retain that distinction. A learner should not be asked to choose a programming language merely to correct character display in an older file.

7.3. Byte offsets and Unicode

The offset array maps Delphi string character positions to original byte positions, including surrogate handling. Range construction uses that map for StartByte and EndByte and records line and column. The terminal offset represents the end of text. Preserve the implemented half-open range behavior when extracting or validating source snippets.

Test multibyte UTF-8, UTF-16, supplementary characters and CRLF/LF boundaries when modifying this code. A naive character index used as a byte offset can make a plausible excerpt point at the wrong bytes while still producing valid JSON.

7.4. Path safety is part of correctness

Inventory.Allowed checks the project root and explicit external roots and rejects reparse-point ancestry through its Windows attribute checks. Source acquisition also resolves file handles to the expected path. Storage destinations apply their own outside-source boundary. These checks protect both confidentiality and the promise that DPH does not modify its selected target.

Do not replace containment checks with a simple unqualified string prefix: sibling paths with similar names and reparse redirection can defeat that approach. Add a focused path regression if a legitimate dependency case requires changing boundary behavior.

8. Project inventory and reading configuration

8.1. Entry files and reachable members

TDPHInventory accepts the selected project entry, configuration, platform, additional defines, explicit external roots and code page. It discovers project XML and Pascal references, captures allowed files, records unavailable files and preserves diagnostics. Sources and ProjectIR are owning dictionaries; SourceDefines records the effective defines for individual files.

Project and group declarations provide settings and references. They do not prove that an output executable exists or a service has been installed. Project-member UI wording must preserve that distinction between a declared target and an observed built artifact.

8.2. Configuration identity and macros

ConfigurationName, PlatformName and ConfigurationIdentity distinguish the selected reading environment. Macro expansion, namespaces, search paths and per-source defines influence what declarations and references DPH can resolve. Additional roots authorize acquisition; they are not a request to recursively scan an entire drive.

SelectionHash in the workspace model includes configuration, platform, defines, roots and code page. A view captured under one selection cannot be treated as current for a different selection merely because source filenames are unchanged. Review and refresh paths compare this identity.

8.3. Unavailable files and incomplete inventories

Missing, excluded or unreadable source must remain represented with qualified availability and diagnostics. Do not drop an unresolved file and then report complete coverage over only the files that happened to open. Inventory.Complete reflects whether the discovery can support its current completeness claim.

For an unreadable dependency, diagnose the entry and reference path, intended configuration and allowed roots first. A new learner still needs useful known material and a focused unanswered question, rather than a wall of internal inventory counters.

8.4. Revalidate before publication

Inventory.Revalidate and CurrentManifest compare the source state after work with the captured input. Source may change while a scan runs. Publication must refuse to make mismatched staged facts current, preserving the last published generation and recording failure or stale state as implemented.

Incremental reuse is safe only when all relevant identity inputs agree. A changed include or binding environment can invalidate routines even when their own body file hash has not changed. The semantic cache includes binding context for this reason.

9. Lexer, preprocessing, and intermediate representation

9.1. Token acquisition

TDPHLexer.Tokenize emits token kind, text, range, active state and condition. It recognizes whitespace, comments, identifiers, numbers, strings, symbols and end-of-file under bounded token acquisition. Malformed strings, comments or numbers require diagnostics rather than an invented valid token stream.

Retaining comments and exact ranges supports later explanations and source evidence. Significant-token accounting is separate from full token count. Do not interpret a high recognized-token ratio as full semantic understanding of the program.

9.2. Conditional preprocessing

TDPHCondition evaluates supported expressions using three-valued active state: true, false or unknown. Both, Either and Opposite preserve uncertainty when a condition cannot be established. The preprocessor handles symbols, directive state and permitted include resolution through a resolver supplied by inventory.

Unknown conditional state must survive into captured conditions and completeness qualifications. Treating unknown as false silently discards a possible implementation; treating it as true silently claims an active path. Add fixtures for nested conditions, defines, includes and malformed expressions when changing this layer.

9.3. TDPHIRNode and TDPHParseResult

The intermediate representation stores Kind, Name, Value, Condition, Parent index, source range and first/last token indexes. TDPHParseResult owns its node list and tokens, recognized/significant/opaque counts and Complete flag. AsJSON records the IR schema and analyzer version alongside the nodes.

Parent is an index into the node collection, not a database entity identifier. Token indexes refer to the parse result's own token array. Preserve those coordinate systems when cloning or serializing; mixing them produces structurally valid but semantically wrong relationships.

9.4. Opaque syntax and diagnostics

Unsupported or incomplete syntax remains qualified rather than being claimed as fully parsed. Opaque regions and diagnostics allow later layers to present known facts while keeping uncertainty. Diagnostics are bounded at 10000 entries in the shared diagnostic collector.

When expanding parser support, add a fixture that demonstrates the previously opaque construct and a negative case that should still diagnose. Do not simply suppress a diagnostic to increase completeness; verify the resulting node structure, ranges and downstream relation.

10. Pascal, forms, and resource parsing

10.1. Pascal declarations and bodies

The Pascal parser recognizes supported program/unit structure, declarations, routines and statement bodies into IR. Declaration and implementation occurrences can refer to the same logical symbol while having different source ranges. Routine parameters and signatures later inform overload and dispatch resolution.

Parser work is a source-reading model, not a substitute for the Delphi compiler. Generics, overloads, conditional code, nested routines and dynamic dispatch require the supported rules and explicit qualifications. Use exact source fixtures to demonstrate a changed resolution rather than relying only on an end-to-end count.

10.2. Form definitions

Forms parsing captures component hierarchy, class names, properties and event bindings from supported DFM/FMX material. A saved event property may connect a component to a handler declaration. Form inheritance and runtime modifications require separate projections and cannot always be reconstructed as the actual visible screen.

Use the saved hierarchy to teach names and relationships. The form preview renderer separates each saved TTabItem into its own design preview and normalizes that tab's offset before painting descendants. An unselected tab is still part of the saved design; IsSelected must not hide it from reconstruction. Label generated form designs as reconstructions. Keep imported screenshots in their distinct channel with contributor, caption and image identity. Replacing that distinction with a generic screenshot caption misstates what was observed.

10.3. Resources and includes

Resource acquisition recognizes supported definitions under bounded reading. Includes retain their source locations and conditional context. Evidence from an included file must refer to that included source and original byte hash, not misleadingly to the top-level unit as if the bytes were there.

Binary or unsupported resource data should remain qualified. A missing form resource may explain why no screen design can be shown, but it does not establish that the application has no screen at runtime.

10.4. Analysis coordination

TDPHAnalyzer creates scans and supports incremental source/parse reuse. TDPHScan owns inventory and results, exposes syntax JSON and ContextHash, and stages source entities and syntax evidence. ContextHash includes analyzer identity, inventory root and configuration identity, manifest identity and per-file defines.

Staging records source metadata, syntax IR and diagnostics as generation facts. The semantic analyzer adds relationships and flow evidence before publication. Keep the staging lifecycle consistent so a later semantic failure cannot leave partial facts labeled as a current complete capture.

11. Symbols, resolution, control flow, and focused analyses

11.1. Semantic model

TDPHSymbol stores canonical key, names, kind, signature, unit and owner, type, condition, variant, dispatch qualification, source hash, range, parameter information and declaration occurrences. TDPHRelation stores source/target indexes, relation kind, resolution, rule, label, condition and evidence range. An unresolved endpoint is represented explicitly.

TDPHSemanticGraph owns symbols, relations, flow nodes, flow edges, diagnostics and source hashes. Its Complete flag is affected by unresolved relations. AsJSON converts indexes to stable external identities and includes schema and analyzer versions, input and context fingerprints, qualifiers and source ranges.

11.2. Routine summary caching

TDPHSemanticAnalyzer maintains FRoutineCache keyed by routine identity. CachedRoutineFacts validates a summary using a hash of binding context, routine key and routine source hash. It stores relation endpoint keys and cached flow/diagnostic information so it can rebind into a new graph rather than reuse stale in-memory indexes directly.

When restoring a summary, flow node indexes are offset into the new graph and endpoints are resolved against the current symbol set. Reuse checks graph limits before appending. A body-text-only cache key would miss changes in imported declarations, overload candidates or binding context.

11.3. Control flow construction

TDPHFlowBuilder builds a graph from statement IR and tokens. Sequence processes statements from the end toward the beginning so each statement can connect to its known continuation. Statement carries explicit next, return, exception, break and continue targets. Branches, loops, case arms and protected regions therefore have different successor rules.

Finally context and label/goto bookkeeping preserve qualifications around nonlocal jumps. Do not replace a try/finally path with a simple sequential arrow. Test early exits, loop exits, exceptions and gotos in fixtures that assert meaningful topology rather than only node counts.

11.4. Bounded graph contracts

The semantic relationship limit is 200000. Flow nodes are bounded at 100000, edges at 300000, and flow-builder nesting at 128. Impact traversal has explicit depth/node bounds. These are implemented limits; UI graph projections may use much smaller bounds to remain readable.

A limit exception must produce qualified failure instead of truncating silently and declaring completeness. A small displayed graph can be a valid bounded view of a larger captured graph, but its omission should not be mistaken for source absence.

11.5. Focused analysis modules

Async examines supported asynchronous patterns. SQL identifies supported statements and data-use evidence. Persistence records source-derived persistence relationships. Quality identifies routine metrics,

potential duplicate bodies, disconnected flow candidates and cycles. These are investigation aids with scope limits, not proof of runtime races, actual database contents, equivalent routines or safely removable code.

The Workspace projections turn these facts into inspectable explanations. Retain qualifiers through rendering and export. If a learner needs to understand a background operation, combine the captured connection with an attributed explanation of its effect and safe expectations.

PART 3: Persistence, authored knowledge, and native presentation

12. SQLite store ownership, migrations, and transactions

12.1. Connection contract

TDPHKnowledgeStore creates a FireDAC SQLite connection for an absolute database path and recorded source root. It applies CheckOutsideSource before opening. Normal locking, disabled shared cache, a 3000 ms busy timeout, foreign-key enforcement and synchronous FULL are configured. The default journal mode is DELETE; WAL is an explicit constructor option and is refused for a UNC database path.

The connection belongs to the creating thread. RequireWriter checks thread identity and lifecycle state. QueueWrite and ProcessQueuedWrites provide the implemented write queue instead of authorizing arbitrary cross-thread connection access. A store is not a thread-safe global service merely because it has a queue field.

12.2. Single writer mutex

A named Windows mutex uses Local\\DPH.Writer plus a SHA256 identity derived from the normalized database path. If another writer already owns that identity, store construction fails with an explicit ownership error. The mutex is released when the store is destroyed.

This guards DPH writers in the implemented local-session namespace. It does not provide a distributed database service or authorize multiple remote writers. Keep error guidance practical: identify and resolve the existing writer rather than increasing busy timeout until the collision becomes less visible.

12.3. Schema migration

CurrentSchemaVersion is 9. ApplyMigrations reads ordered SQL files, using the explicit DPH STATEMENT separator to execute migration units. PRAGMA user_version prevents an older application from opening a newer unsupported schema. Foreign keys are verified as active, and integrity is checked after migration.

Migrations define constraints, indexes and triggers in addition to tables. Do not copy only CREATE TABLE definitions into a new database. A migration change requires an old-store opening test, a failure/rollback test where appropriate and preservation of authored revisions. Keep applied migration semantics stable; introduce a new numbered migration for a new persisted contract.

12.4. Interrupted generation recovery

On store startup, running generations left by an interrupted writer are diagnosed and failed, and project stale/failure state is updated transactionally. The application must not assume an unfinished generation became usable merely because some rows or a cached file exist.

Retain the last published head as the reading baseline when a new scan fails. An engineer diagnosing interruption should inspect generation status, diagnostics and project_state before considering repair. Avoid manually promoting a running or failed generation to current.

13. Generations, source identity, and cached views

13.1. The persisted graph of records

projects is a singleton project record. configurations stores reading environments and their fingerprints. analysis_generations associates each capture with its project, configuration, manifest and analyzer. project_state records the published head and independent stale, failure, modification and knowledge-review flags.

source_files identifies canonical relative paths and observed/analyzed identity. file_revisions and generation_files associate bytes and availability with particular captures. entities, entity_versions, evidence, relationships and flow tables preserve facts within generation/project boundaries. Composite foreign keys keep records from different projects or captures from being linked accidentally.

13.2. Stage commit and publish are separate

BeginGeneration creates the running capture. BeginFacts starts staged fact work. CommitFacts persists staged content; RollbackFacts abandons a failed fact transaction. PublishGeneration checks the revalidated manifest and completeness before switching the published head. FailGeneration records failure while preserving the earlier usable head.

An engineer should identify the exact transaction boundary before modifying scan flow. Committing rows is not equivalent to publishing them. UI code must not bypass PublishGeneration by loading whatever generation has the largest numeric ID.

13.3. Stable identity and changed objects

Canonical entity keys identify logical source objects under the implemented signature/structure rules. A name or signature change may create a new identity rather than automatically moving human knowledge. ContinuityCandidates and ApproveContinuity support explicit review of proposed replacement links, retaining the history.

Do not assume text similarity proves the same logical object. Inspect kind, scope, signature, structural evidence and linked explanation. An approved continuity link preserves traceability; the knowledge content still needs review for current correctness.

13.4. Cached workspace views

SaveWorkspaceView persists a captured JSON view in managed storage and associates it with the appropriate generation. LoadWorkspaceView retrieves that view for later reading. A cache is a projection of immutable facts and selected capture context, not the authoritative source of authored revision history.

Opening a saved project verifies the cached artifact and compares its captured manifest fingerprint, analyzer and discovery versions, and reading selection with the attached source. A partial capture remains marked stale for interpretation, but that flag alone must not cause an identical expensive scan on every open. Explicit reassessment, changed inputs, versions, or selection start a new scan. Preserve the partial qualification in the learner interface and exported evidence.

Retention removes eligible older cached views while preserving protected generations and linked evidence under the implemented policy. It does not delete the immutable analysis fact history or human knowledge merely because an older view file is eligible. Backup must include managed images and observations, not only cache JSON.

13.5. Relocation contract

RelocateSource updates a source root only after verifying the intended manifest and generation. A root mismatch on ordinary store open is refused. This prevents a convenient database filename from attaching knowledge to unrelated source.

Test relocation with identical intended source, changed source and an unrelated project. Preserve the previous store on refusal. Avoid rewriting root_uri manually to get past a check; it is part of the project's identity and evidence boundary.

14. Authored revisions, links, and review state

14.1. SaveKnowledge contract

SaveKnowledge accepts knowledge identity, expected revision, source generation, title, body, editor, kind, entity association, document destination, audience, links, question metadata and structured JSON through its implemented signature. It validates inputs and appends a revision rather than overwriting history. The caller must supply the revision it actually edited.

Optimistic revision checking prevents a later save from silently replacing newer content. A conflict is a data reconciliation event, not a reason to discard the expected revision parameter. The UI should preserve the draft, refresh the current record and allow deliberate reconciliation.

14.2. Structured text and typed metadata agree

Migration 009 adds structured_json to immutable knowledge revisions with a JSON-object constraint and default {} for older material. Task, teaching-card, profile, glossary and other supported types have validators and canonical body renderers. The stored text and structured record must agree under the corresponding save rules.

This prevents an editor from storing one human-readable procedure while typed metadata drives a different task reader or export. When adding a typed record, implement validation, canonical body construction,

immutable identity checks and relevant projection tests together. Preserve legacy unstructured notes rather than inventing fields for them.

14.3. Links have precise targets

Knowledge links associate revisions with source entities, evidence or other exact records under project/generation constraints. Rich links preserve target kind and identity. Document sections and audiences determine where supplied knowledge is projected. A global unlinked note must not become the purpose of every source object.

Link validity and review status are separate. A valid source key proves that the link resolves under the captured view; it does not prove the contributor's interpretation. An excluded target in publication must not leave a leaked private detail or a misleading active link.

14.4. Review lifecycle

ReviewKnowledge records a named decision against the expected revision and relevant generation, with current-source prerequisites where required. Save creates pending attributed content. Verified review, rejection and retirement are separate state transitions; historical revisions remain available.

Current teaching-card and task projections exclude rejected and retired content. Review-needed state can arise from source changes even when the explanation text is unchanged. Test both positive projection of current material and exclusion of superseded/rejected/retired material.

14.5. Document sections and questions

Document-section metadata organizes knowledge for overview, user, engineering, maintenance, build/deploy, troubleshooting, orientation, handoff and help destinations. Audience is separately scoped. Author-question metadata preserves the original prompt and answer relationship rather than treating a reply as unrelated free text.

Human review is not a parser function. Keep unresolved questions visible in beginner language, and make it possible for a contributor to add an answer with context and a named owner. Do not promote an answer because its associated source file parsed successfully.

15. Structured tasks, teaching cards, and personal progress

15.1. Learning lesson schema

A learning_lesson has a canonical GUID lesson_id, six fields named name, purpose, prerequisites, expected_result, troubleshooting and practice, and an ordered steps array. Each step contains action and expected_result. A backward-compatible optional learning object contains exactly role, path and order.

Validation requires one through 100 steps, nonblank supported fields, a descriptive task name no longer than 200 characters, field and step text within 4096 characters, and complete structured JSON within 65536 characters. Step action and task purpose cannot be a meaningless Unknown placeholder. The optional role/path strings are each bounded at 200 characters and order must be an integer from 1 through 10000.

15.2. Task editing and reading states

The editor holds FTaskDraft separately from the step currently being edited. New step creates a draft step, Keep step accepts it, Undo step edit abandons only that edit, and Move up/down reorder the draft array. Save task validates and persists the complete canonical record asynchronously. Cancel changes abandons the draft explicitly.

The reader filters by search, role and path, sorts prepared tasks by path/order/title and presents one step at a time. Legacy unstructured user-help notes remain readable without invented steps. Other typed records must not accidentally appear as task walkthroughs merely because their kind resembles user_help.

15.3. Valid task metadata example

The following minimal typed record illustrates the persisted teaching contract. Its values are sample instruction, not SQL commands. Use LearningLessonBody to derive its canonical body and the normal save path to persist it with contributor, kind, revision and generation metadata. The typed record alone is not a complete knowledge database row.

```
{
  "record_type": "learning_lesson",
  "lesson_id": "{59A27D12-66A0-48EA-A5AF-DB41CDB33EE8}",
  "fields": {
    "name": "Find a customer record",
    "purpose": "Locate a record from an approved search screen.",
    "prerequisites": "Read the selected application's purpose.",
    "expected_result": "The matching record is shown.",
    "troubleshooting": "Check the search value and ask the owner about access rules.",
    "practice": "Explain which result identifies the intended record."
  },
  "steps": [
    {
      "action": "Enter an owner-approved search value and choose Search.",
      "expected_result": "A matching record appears in the result list."
    }
  ],
  "learning": {"role": "New learner", "path": "Customer lookup", "order": 2}
}
```

15.4. Teaching card schema

A learning_card contains canonical card_id, one of twelve categories and exactly seven fields: name, role, meaning, example, limits, help and practice. Optional picture metadata contains image_sha256 plus normalized x and y coordinates. Canonical body generation preserves the same content for general knowledge, the native library and offline lessons.

Card identity and type are retained during revision. The storage layer checks uniqueness and canonical text. The reader filters title/body and role, keeps All learners material available, excludes rejected/retired records and resolves contextual links from both links and rich_links to current captured symbols.

15.5. Valid teaching card metadata example

Use LearningCardBody for the matching canonical text. A picture callout adds the optional picture object only when image_sha256 identifies an imported image in this project. The numeric coordinates are normalized integers, not display pixels.

```
{
  "record_type": "learning_card",
  "card_id": "{FC3FD5CC-C050-4E22-BBDC-4A760063CE49}",
  "category": "Field",
  "fields": {
    "name": "Quantity represents whole items",
    "role": "All learners",
    "meaning": "Enter the number of whole items to price.",
    "example": "Quantity 2 at unit price 10.00 gives subtotal 20.00.",
    "limits": "Use a whole number from 1 through 1000.",
    "help": "Replace a fractional quantity with a valid whole number.",
    "practice": "Why is 0 invalid? It is below the minimum of 1."
  }
}
```

15.6. Progress key and semantics

TDPHWorkspaceCatalog.LearningProgress and SetLearningProgress key progress by database, learner, lesson identity and exact knowledge revision. Learner names are trimmed and bounded from 1 through 120 characters. Checked-step counts are bounded from 0 through 100. Persistence uses the existing atomic catalog save path rather than adding progress to immutable lesson content.

I checked this result records consecutive checks and advances to the next unchecked step. Next/Previous are reading navigation. Start again changes the reading position without clearing checks. Clear my progress resets the selected learner/task/revision key. A revised task does not inherit old progress automatically.

15.7. Progress is not authentication or mastery

Role filters are teaching metadata. Learner names are local keys. Neither provides authentication, target-system permission or a certification result. UI wording and exported prose must not imply that checked-step count proves successful target execution or competence.

Test persistence after changing tasks and reopening, distinct learners, changed revisions, reset behavior and invalid count/name rejection. A test that only checks incrementing an integer misses the identity and stale-revision contracts that make progress useful.

16. Pictures, artifacts, and anchored callouts

16.1. Import and managed storage

ImportScreenshot accepts generation, existing PNG file, author, caption, optional source entity key and image kind. The default kind is Screenshot; Sample output and Other teaching image preserve the educational distinction. Imported images are managed artifacts associated with the selected project and captured context.

Validate supplied image format, size, identity and outside-source storage according to the implementation. Do not accept an arbitrary file extension as proof of PNG content. Import does not launch or capture the target application. A caption should state what the contributor supplied and which context it illustrates.

16.2. Normalized coordinates

Picture metadata identifies the managed image by SHA256 and stores x/y in the normalized range 0 through 10000. Normalization makes a point independent of the displayed pixel size. The native paint path multiplies that coordinate by the current image extent; offline HTML uses corresponding relative positioning.

At import/save time, the image must exist in this project's managed image set. A syntactically valid hash is insufficient if it names another project's image or no image. The original PNG bytes remain unchanged; markers are an overlay, not destructive raster edits.

16.3. Native style and interaction

ScreenshotImage hosts ScreenshotMarkers, a serialized TPaintBox with hit testing disabled so the underlying image receives point selection. ScreenshotMarkerStyle and ScreenshotMarkerText are serialized style prototypes. ScreenshotPoint records the selected normalized location; Explain this point creates a teaching-card draft tied to that image point.

ScreenshotMarkerPaint draws numbered markers using the current image extent, while the legend presents attributed teaching content. The serialized ScreenshotModeCombo starts in Native pixels; Fit and custom zoom change the display transform. The lower evidence memo uses a compact height and does not repeat the help text above the image. Test markers near boundaries and after zoom/resize so a correct stored point does not become a misplaced visual label.

16.4. Offline presentation

The publication renderer includes the supplied image and proportional marker overlay with a numbered explanation legend. The same meaning, example, help and practice content should survive into learning.html. Redaction/scoping must account for image captions and linked knowledge even though an image's pixels cannot be fully sanitized by text redaction.

Test both byte preservation and instructional meaning. A checksum-only test proves the image was copied, but not that a learner can connect marker 1 to the right worked example. A native-only visual test misses export placement and caption loss.

17. Learner projections and native binding

17.1. TDPHWorkspaceView

The workspace view owns a JSON capture and a symbol lookup dictionary. Symbol, Neighbors, Search, Hierarchy, Properties, Details and Graph expose focused projections. Capture combines scan, graph and selection context into a reusable document. Coverage and Questions report separate qualifications rather than erasing incomplete evidence.

Methods return JSON values with explicit ownership conventions. Follow the caller's finally/free patterns and avoid retaining borrowed objects after the view is replaced. A stable external key is safer for selection state than a pointer into an old JSON document.

17.2. Handoff dossier and legacy explanations

TDPHHandoffDossier.Render produces four native handoff pages from the selected, locally displayed document model: Handoff overview, Commands and workflows, Data and operations, and Risks and open questions. The renderer bounds each list, escapes source text, links to captured objects and records, and marks unreviewed or stale facts. It presents form registration and event bindings as static evidence, not observed user workflows. DPH.UI.Main.fmx serializes the native page labels so they remain IDE-editable.

TDPHExplanationModel.ApplicationRouteHTML still produces the older four-part learning edition used by export and several specialist views. It is retained for compatibility beside the exported handoff pages. Its authored tasks and teaching cards remain accessible; the Handoff dossier does not reclassify them as verified behavior. ObjectLessonHTML introduces role and relevant purpose, actual conditions and bounded next connections, then keeps the deeper code walkthrough optional. It must exclude rejected or retired content and preserve unresolved questions.

17.3. Lazy optional reports

WorkspacePageChange chooses the active projection. Report generation for hidden specialist tabs should not delay opening the Handoff dossier. Refresh routines must distinguish loading the base view from building graphs, generated form previews, quality reports or screen-history comparisons on demand.

When adding a learner destination, define what question it answers and reuse the existing model. Do not expose every serialized tab simply to make the selector appear comprehensive. A learner's information hierarchy and the repository's technical module hierarchy serve different readers.

17.4. Layout state and compact windows

The main form opens maximized and enforces a minimum client size of 960 by 600. Side panes appear for relevant object work while focused readers use the available width. TaskContextExpander collapses prerequisites/context to preserve room for the current action and expected result.

Validate native labels, scroll ranges, tab order, buttons and text areas at both maximized and restored sizes. A successful screenshot of an HTML page does not validate a native editor. Inspect how a user reaches commands that require horizontal scrolling and document that behavior explicitly when necessary.

PART 4: Publication, search, and operational safety

18. Publication model and export contract

18.1. Canonical scoped model

TDPHDocumentModel receives workspace view, knowledge, seeds, exclusions, detached/stale state, images, display mode and audience. It builds a schema-versioned document with project/capture identity, analyzer

versions, reading configuration, source revision qualification, symbols, evidence, relationships, flow, sections and knowledge. The current learning template version is DPH-LEARNING-13.

Publication audiences include all, engineer, user, maintainer, builder and agent under validation. Local display can preserve original local content, while export uses scoped redacted values. Do not use an unscoped native view directly as an export model.

18.2. Scoping before rendering

Allowed files and selected keys are built before renderers run. Seeds define selected-object traversal; exclusions remove specified files or prefixes. Related evidence, flow and knowledge links must be checked against the retained scope. Excluded source must not leak through a secondary link, comment, trace or image caption.

Test both the obvious omitted page and indirect references. A renderer that simply hides one table after serializing an unredacted JSON document still leaks the content in another output format. The canonical model is the common privacy boundary.

18.3. Redaction and encoding

Redact masks supported secret patterns such as private-key blocks, bearer tokens, credential assignments and recognized access-token shapes. Scrub recursively handles JSON fields and converts structured_json into structured_record for exported typed metadata. HTML encoding converts contributed text to text content rather than trusting it as markup.

Pattern redaction is a safeguard, not a guarantee that arbitrary images or prose contain no confidential data. Review the intended edition before sharing. Keep product identifiers and code names appropriately protected from optional page translation where used.

18.4. Export files and atomic destination

TDPHPublisher.Export writes a new destination and refuses an existing one. The four handoff-N.html pages are generated from the scoped publication model and link to included object pages and the technical index. learning.html retains the earlier four-part learner route, authored tasks, examples, questions, terms and supplied images. The index and README direct recipients to the handoff first. Hash-named object pages provide linked explanations with optional source detail; JSONL technical indexes support deeper inspection and validation.

The edition also includes its manifest/provenance, checksum index and document companions under the implemented export structure. Optional source navigation and generated agent instructions are explicit options. Preserve fresh-destination behavior so a failed or changed export does not silently mix files from two captures.

18.5. Validation trust boundary

TDPHPublisher.Validate checks filenames, hashes, canonical provenance, ranges, technical indexes and evidence agreement against a separately supplied trusted input fingerprint. The command-line tool requires bundle-directory and trusted-input-fingerprint. Its optional --allow-stale flag permits historical/partial inspection without making the material current or complete.

A checksum supplied inside an unknown bundle is not author authentication. The separately retained expected identity is essential to the integrity claim. Do not implement a repair that rewrites a failing bundle's checksums and calls it verified without regenerating from the trusted source model.

19. Search, browser navigation, and supporting integrations

19.1. Persistent project search

BuildSearchIndex creates generation-scoped searchable material for objects, evidence, knowledge and diagnostics. SearchProject validates the query, result kind and page offset. The UI explains the contract: up to twelve words, prefix matches and all words required, with pages of 50 results.

Keep search excerpts qualified by their result kind. A matching diagnostic is not a reviewed teaching explanation. Refresh or generation switches must not apply stale result pages to a new workspace. Test missing terms, Unicode, filtering, pagination and source/knowledge results with meaningful assertions.

19.2. Embedded browser policy

DPH.Publish.BrowserPolicy constrains navigation to expected local roots and supported routes. BrowserShouldLoad inspects requested URLs before navigation and routes supported internal object actions through selection handlers. Generated HTML links must remain inside the edition or intentional policy rather than loading arbitrary file paths.

Path traversal, encoded separators, untrusted schemes and outside-root file navigation belong in browser-policy tests. A page that renders correctly can still contain unsafe links. Never use HTML rendering success as evidence that navigation policy was applied.

19.3. Runtime observations and data-quality imports

Imported observations are separately attributed records tied to supported project/generation context. DPH does not infer that a source analyzer executed the target. Data-quality imports similarly represent supplied evidence and explicit analysis limits. Their JSON/size/path validation remains necessary even when a page is not on the main learner route.

For a lesson about a real background task or error, distinguish the supplied observation from static evidence and author interpretation. Exports must preserve that attribution and qualification. An imported report is not authority to connect to the target database or execute diagnostic scripts.

19.4. Translation dependency

The repository pins DelphiAppTranslation under dependencies with its license and snapshot identity. Translation cache and adapter code preserve source revision, locale and review information. Optional HTML help translation uses the approved GTranslate widget and local selector helper, requiring connectivity for the remote widget.

Do not conflate optional help translation with source text decoding. Do not assume the existence of translation backend code means a particular remote provider is configured or an ordinary translation page is exposed. Inspect the current UI and adapter configuration before documenting a workflow.

PART 5: Building, testing, diagnosis, and release

20. Build environment and source layout

20.1. Required engineering tools

The project uses RAD Studio 37.0 for the installed Windows toolchain. rsvars.bat initializes BDS and compiler/library environment. dcc32 builds the current Win32 application and console tests, while the application project build script invokes MSBuild with Debug and Win32. Use the configured RAD Studio path in your environment rather than copying a personal workstation path into end-user directions.

The repository uses FireDAC SQLite, FireMonkey, Windows APIs and the pinned DelphiAppTranslation source. The WebView2 loader is supplied under tools/WebView2/Win32. Microsoft WebView2 Runtime remains a system prerequisite. LibreOffice renders editable guides and creates PDF companions; the guide scripts use Python with python-docx and pypdf plus Poppler for visual inspection.

For the standard installed RAD Studio location, initialize a command prompt with the command below. This is a tool installation path, not a user project directory. If the toolchain was installed elsewhere, adjust this location and the build scripts consistently.

```
call "C:\Program Files (x86)\Embarcadero\Studio\37.0\bin\rsvars.bat"
```

20.2. Directory responsibilities

Project Pascal units and .fmx resources are at the repository root. bin/Win32/Debug contains executables and dcu contains compiled units. docs/guides contains current editable guides and directions; docs/pdf contains PDF companions. Original specifications are in docs/planning, the requirement ledger is in docs/requirements, JSON checks are in docs/validation/reports, test pictures are in docs/validation/images, and logs are in logs/validation or logs/development. help contains local HTML, export contains desktop or lesson deliverables, and source distributions contains versioned source archives. docs/PROJECT_LAYOUT.txt and docs/PROJECT_MOVE_MANIFEST.csv record the complete folder map and every moved file.

samples holds developer fixtures and optional locally pinned reference material; OrderPractice remains a parser/form fixture without imported learner lessons. tools holds build, packaging, documentation and validation tools. migrations holds the ordered SQL schema. dependencies holds pinned third-party source. Databases is the default suggested lesson-store area; user stores must not be committed or packaged.

20.3. Build the application and form baseline

1. Open an engineering command shell in the repository root with the privileges required by the configured RAD Studio environment.

2. Run tools\build-wp0.cmd. It initializes the RAD environment, creates the configured output directories, copies the WebView2 loader and invokes the project build.

3. Allow its form-baseline test to run. Read both the compiler result and the final form-instantiation PASS line.

4. Confirm that bin\Win32\Debug contains DelphiProjectHandoff.exe and the loader. Start the application from that location for a development inspection.

5. If the executable is locked, identify the exact running project process and confirm it has no unsaved work before closing the permitted project instance and rebuilding.

Expected result: current source compiles and the serialized main form binds correctly. Compiler warnings should be reviewed in context; do not hide a new diagnostic by discarding all warnings.

20.4. Build the publication validator

tools\build-wp7.cmd compiles DPH.ValidateHandoff and the hardening test, then runs the latter. tools\build-glossary.cmd also compiles the validator after its glossary checks. If only recompiling the validator, use the same initialized RAD environment and the compiler flags declared by those scripts.

The validator shares the current publication units. Package a validator compiled against the same source contract as the exported template. A stale validator executable beside a newer application can produce misleading deployment checks.

21. Meaningful test selection and execution

21.1. Select tests by the changed contract

Form layout or event-binding changes require the baseline and relevant native UI checks. Task/card changes require GuidedTasks, Catalog and the selected-application learning test. Knowledge persistence changes require Knowledge and Storage. Publication changes require Publishing and the applicable hardening/privacy checks. Analyzer changes require their focused parser/semantic fixtures and relevant reference/incremental checks.

A test should demonstrate observable contract behavior. Checking that a function returns a nonempty string is insufficient for a teaching explanation that must preserve meaning, attribution, order and safe recovery. Do not add repetitive tests that merely mirror an implementation assignment.

21.2. Current learning test commands

```
tools\build-wp0.cmd
tools\build-learning-refocus.cmd
tools\build-guided-tasks.cmd "<new disposable task workspace>"
tools\build-order-sample.cmd
tools\build-wp6.cmd
```

Read the script before using an optional argument. Use a fresh disposable workspace outside source fixtures when a test writes a database or exported edition. Some scripts accept a reference database; others create their own temporary paths. A command's mere presence in this list is not authorization to run it against an owner store.

The catalog test is tests\DPH.Tests.Catalog.dpr; it does not have a dedicated build-catalog.cmd wrapper. Compile it with the same initialized dcc32 environment and unit search paths as build-guided-tasks.cmd, then run its executable with a disposable root directory as its first argument. It creates bin\catalog-validation under that root; supply a dedicated temporary root so this does not replace a user's workspace.json. Inspect its ParamStr calls before invocation.

21.3. Selected-application learning integration

DPH.Tests.LearningRefocus creates a disposable selected-project capture, verifies that the main learner views use that project's own forms and action links, and confirms that no OrderPractice lessons are imported. Its native WebView2 path checks the route title, internal links, text fallback and compact layout. The test writes screenshots into its disposable workspace and publishes selected current-interface captures for the manuals only after all checks pass.

DPH.Tests.GuidedTasks checks structured task validation, ordered steps, revision-specific progress, teaching cards, picture callouts and publication of selected-app instructions. DPH.Tests.OrderSample still tests a developer fixture's calculation behavior; it is not a default learner path. None of these tests proves that an arbitrary selected application ran or that its unwritten business rules are known.

21.4. Full and incremental reference checks

The pinned reference project provides a larger real source set for analysis, cache, export and incremental consistency. Preserve its bytes and fixture identity. Reusing one prepared reference cache can avoid generating many duplicate test folders without weakening the relevant comparison.

Compare semantic identity and meaningful facts between full and incremental analysis, not only elapsed time or counts. A routine-cache change requires a fixture where binding changes without an equivalent body edit. A form event change requires checking the actual linked handler and source range.

21.5. Isolate local preferences and artifacts

Native tests may use the ordinary per-user workspace catalog. Preserve and restore its original bytes when the test cannot use an isolated path. Avoid testing against a user's actively edited DPH instance. Keep generated stores, exported editions and temporary compiled sources under one explicitly owned validation area.

After success, retain concise logs and relevant screenshots, then remove only verified disposable directories created by that validation phase. Check containment and reparse points before recursive deletion. Never delete owner databases, source fixtures, untracked documents or a reference source tree as generic test cleanup.

22. Package, deploy, and verify extracted copies

22.1. Binary package contents

The 0.8.0.3 experimental desktop package contains the Win32 and Win64 Release DelphiProjectHandoff.exe builds, a WebView2Loader.dll beside each, offline language packs, the German trial model, migrations, help, current guides, directions, DPH's Apache 2.0 license, and third-party notices. It does not include an owner .dphdb, private document copies, or reference-project working stores. A previously created package is a historical build and must not be presented as the current executable. The unmodified Bergamot model is attributed under CC BY-SA 4.0, with its license text and indexed file hashes included in the package.

The editable source distribution is the primary public delivery. It includes the project and forms, source, pinned translation dependency, guides and their source and illustrations, tests, and the OrderPractice example. That example is for reproducible development checks; it is not an imported lesson for an arbitrary selected application.

22.2. Run the packager deliberately

1. Complete the code and document validation and build current Win32 and Win64 Debug and Release executables.

2. Review the exact source files and guide illustrations to distribute. Run tools\package-source-distribution.ps1 with a fresh version value, including four-component experimental versions such as 0.8.0.3. It verifies every source file hash and refuses an existing archive name.

3. Extract the source ZIP into a fresh directory and build it. Check that the example, documentation source, 35 language packs, form resources, and guide illustrations are present.

4. If a ready-to-run experimental Windows package is also offered, run tools\package-dph.ps1 with the same version after the source and executable agree. Verify its extraction, hashes, and launch separately. Never label an older binary ZIP as the current source edition.

Expected result: a fresh versioned delivery exists. Do not overwrite an earlier release's files and keep its old checksum report as if it still described the new content.

22.3. Verify the binary extraction

Extract a newly made binary ZIP to a new writable location. Verify every SHA256.json entry against extracted bytes. Check required migrations, current manuals, local help and loader. Start the extracted executable, confirm a responsive native window and normal close, then select a disposable Delphi project and verify that Handoff dossier names it, labels uncertainty, and links to source. Check that the other five tabs open their findings and that Screenshots generates previews when selected.

Do not test the delivery solely by running the development executable. That can conceal missing package files because the source tree supplies them. After adding temporary verification tools to an extracted copy, keep that copy separate from the pristine release staging folder.

22.4. Verify the source extraction

Extract the source ZIP into a different new directory and run its application build/baseline there. This demonstrates that source, forms, project file, dependencies and tools required for the build are actually included. Preserve the build log and exact archive hash. Check the source ZIP's SHA256.txt entries against extracted files.

The source archive intentionally excludes personal owner data and locally pinned reference source. A reference-only test may require separately supplied reference material, while the ordinary application and included developer-fixture tests must remain reproducible from shipped source.

22.5. Release record and repositories

Review git status, stage only intended changes and artifacts, commit with a concrete description and push to every configured public/private remote under the project's authorization. Verify each remote branch points at the intended commit. Do not force-push or overwrite unreviewed remote work to simplify synchronization.

Keep release validation, source provenance and manual version evidence together. If a guide is corrected after a release, deliver clearly identified companion guides or a fresh package version rather than silently changing a historical ZIP. A user's untracked PDF copies are owner documents, not automatically release artifacts.

23. Diagnose failures at their owning layer

23.1. Acquisition and inventory failure

Record the entry path, source-reading configuration and first diagnostic. Check allowed roots, referenced filenames, encoding and missing include/form dependencies. Inspect the acquisition method's original-byte range and path checks before modifying parser behavior.

Do not broaden acquisition to unrelated directories merely to eliminate missing-file diagnostics. If the target genuinely depends on external source, obtain an explicit dependency root and preserve its identity in configuration and manifests.

23.2. Parsing or semantic mismatch

Reduce the failing construct to a protected fixture. Inspect tokens, conditional active state, IR node parent/ranges and the semantic resolution rule. If full analysis is correct but incremental differs, inspect context hashes and cache rebinding before patching the renderer.

Keep unsupported syntax qualified. A new parser branch must improve actual structure and downstream facts, not just suppress opaque-token accounting. Check Unicode range behavior if the excerpt or line target is wrong despite a correct object name.

23.3. Store open or migration failure

Check the exact schema-version, integrity, source-root or writer-ownership error. Preserve the database and managed folder before repair. Inspect PRAGMA user_version, migration history and relevant foreign-key constraints using an approved diagnostic copy.

Do not disable foreign keys, delete history rows or overwrite root_uri to make the store open. Those actions can hide cross-project links or corrupt provenance. A newer-schema store requires a compatible build or a deliberate tested migration plan.

23.4. Save conflict or lost draft report

Reproduce with the exact knowledge identity and expected revision. Compare the submitted snapshot with the current editor and the response branch clearing FKnowledgeEdited. Inspect stable task/card saving IDs and ownership transfer when the list reloads.

For a conflict, verify that the original draft remains recoverable and the latest saved revision is not overwritten. For a canonical-body error, compare typed fields with the renderer used for that record type. Do not relax equality merely to allow two representations to diverge.

23.5. Export or navigation failure

Rebuild a scoped model and examine the failing filename, hash, range or link. Inspect exclusions, audience and transformed structured metadata before editing HTML. Run the validator with the separately trusted expected fingerprint. If a browser refuses a link, inspect BrowserPolicy and the generated URL's decoding/containment behavior.

Never repair a broken trust boundary by permitting all file URLs or regenerating checksums over unexplained mutated content. Fix the producing model or route, then generate a new edition and validate it.

23.6. Native layout or browser display failure

Separate native resource/binding failure from embedded-browser failure. The baseline test covers the former structurally; actual restored/maximized inspection covers usability. WebView2 Runtime, loader and generated-page existence matter for the latter.

Check scroll ownership, alignment, margins, collapsed context and reader height at the minimum size. The user must be able to find the action, its expected result and the save/navigation commands without guessing which unlabeled area to scroll.

24. Maintain documentation as an engineering artifact

24.1. Source for the current manuals

tools\build-guides.py builds the current DPH manuals from the authored User and Engineering source text in docs\guides\source, supported native images and generated technical references. It records source hashes in docs\validation\reports\guide-source-validation.json. These manuals describe DPH; the selected application's published guides are generated through DPH.Publish.Export and are a separate output path.

Keep control names aligned with .fmx resources and event handlers. Documentation must distinguish a currently accessible selector route from a backend method or hidden serialized legacy page. New instructions should begin at an explicit visible starting point and finish with a result the reader can check.

24.2. Editable contents and companion PDFs

The manuals use real heading styles with numbered parts, chapters and subsections. The contents is an editable field, with a cached page-number result populated against its matching rendered PDF. tools\finalize-guide.py updates that cached result without destroying the field. tools\render-guides.ps1 uses LibreOffice and the document renderer to produce PDFs and page PNGs, then rerenders after contents population.

If page numbers move during field population, repeat verification until cached entries agree with the final PDF. A table of heading names without valid page numbers is not a finished contents. A printed manual needs consistent running headers, page numbers and readable hierarchy as well as searchable text.

24.3. Review depth and instructional completeness

For the User Guide, verify that a first-time reader can build and open DPH, select an unfamiliar application, follow its starting point, read a project document, inspect a source-linked object, examine a generated blank screen, understand a diagram, and reopen the saved .dphdb. Keep screenshots from the current DPH build and label form reconstructions accurately. Do not direct readers to hidden legacy authoring pages.

For the Engineering Guide, verify the explanation of ownership, source ranges, configuration, parser/semantic boundaries, generation transactions, migrations, authored revisions, structured schemas, progress identity, image coordinates, privacy, build/test commands and extracted-delivery validation. File inventories support this explanation; they do not replace it.

24.4. Visual and privacy checks

Inspect every final rendered page for clipping, stranded headings, excessive blank space, broken code wrapping, unreadable figures and contents errors. Reopen the DOCX to verify field and heading structure and reopen the PDF to verify pages and text. Retain evidence of the exact final files reviewed.

Remove personal author metadata, development-machine references and historical conversational disputes from both manuals. Standard tool versions and relative repository paths belong in the engineering text where useful. User instructions should refer to the recipient's extracted delivery, not a particular developer's machine.

PART 6: Technical reference

25. Source units and public contracts

25.1. Reading the generated unit reference

The following reference is generated from the exact current Pascal source files. Each entry identifies the layer responsibility, current file size and public type or routine declarations. Read these declarations alongside the architectural chapters before changing their implementation. Private routines and implementation details remain in the named unit.

The reference intentionally includes supporting and legacy projection modules so an engineer can understand the existing repository. Their inclusion does not add those pages to the learner interface or claim unrestricted semantic capability. Public declarations show callable contracts; the implementation and associated tests establish behavior and limits.

25.2. Analysis.Async

Source file: DPH.Analysis.Async.pas. Adds qualified evidence for supported asynchronous source patterns. Static patterns are not runtime scheduling observations.

Principal public types: TDPHAsyncAnalysis.

```
class procedure Enrich(AScan: TDPHScan; AGraph: TDPHSemanticGraph);
```

25.3. Analysis.Explanations

Source file: DPH.Analysis.Explanations.pas. Builds learner topics and object explanations from source facts and relevant attributed knowledge. Keep meaning first and deeper code optional.

Principal public types: TDPHExplanationModel.

```
constructor Create(AFacts: TJSONObject);
destructor Destroy;
class function Encode(const AText: string): string;
function ObjectText(const AKey: string): string;
function ObjectHTML(const AKey: string): string;
function ObjectLessonHTML(const AKey: string): string;
function RoutineLesson(const AKey: string): string;
```

```
function TopicText(AIndex: Integer): string;
function TopicHTML(AIndex: Integer): string;
function LessonHTML(AIndex: Integer): string;
function ApplicationRouteHTML(AIndex: Integer): string;
function ProjectMapHTML: string;
function FlowSVG(const AKey: string): string;
function FlowHTML(const AKey: string): string;
function GraphHTML(AGraph: TJSONObject): string;
function TraceHTML(const AKey: string): string;
class function TopicTitle(AIndex: Integer): string;
class function ApplicationRouteTitle(AIndex: Integer): string;
```

25.4. Analysis.Flow

Source file: DPH.Analysis.Flow.pas. Constructs statement control-flow topology with explicit continuations, exceptions, returns, breaks and finally context. Preserve branch semantics.

Principal public types: TDPHGotoLink, TDPHFlowBuilder.

```
procedure Build(AGraph: TDPHSemanticGraph; ARoutine: Integer; AIR: TDPHParseResult; const
ATokens: TDPHTokens; ABody: Integer);
```

25.5. Analysis.Model

Source file: DPH.Analysis.Model.pas. Owns semantic symbols, relations and control-flow data plus provenance. Unresolved relations and bounded traversal remain explicit.

Principal public types: TDPHSymbol, TDPHRelation, TDPHFlowNode, TDPHFlowEdge, TDPHSemanticGraph.

```
constructor Create;
destructor Destroy;
constructor Create;
destructor Destroy;
function AddRelation(AFrom,ATo: Integer; const AKind,AResolution,ARule,ALabel: string; const
ALocation: TDPHRange; const ACondition: string = ''): Integer;
function AsJSON: TJSONObject;
function ReverseReferences(const AKey: string): TJSONArray;
function Impact(const ASeeds: TArray<string>; AMaxDepth: Integer=32; AMaxNodes:
Integer=10000): TJSONObject;
function SymbolIndex(const AKey: string): Integer;
function DPHRangeJSON(const ARange: TDPHRange): TJSONObject;
```

25.6. Analysis.Persistence

Source file: DPH.Analysis.Persistence.pas. Persists supported semantic facts and their references into generation-scoped storage. Keep fact transactions and publication separate.

Principal public types: TDPHSemanticPersistence.

```
class function Stage(AScan: TDPHScan; AGraph: TDPHSemanticGraph; AStore: TDPHKnowledgeStore;
AConfiguration: Int64): Int64;
class function Publish(AScan: TDPHScan; AGraph: TDPHSemanticGraph; AStore: TDPHKnowledgeStore;
 AGeneration: Int64): Boolean;
```

25.7. Analysis.Quality

Source file: DPH.Analysis.Quality.pas. Projects metrics and investigation candidates from captured facts. Candidate findings do not prove unused code or safe removal.

Principal public types: TDPHSourceQuality.

```
class function Analyze(AScan: TDPHScan; AGraph: TDPHSemanticGraph): TJSONObject;
```

25.8. Analysis.SQL

Source file: DPH.Analysis.SQL.pas. Adds supported SQL/data-use evidence from source. It is not permission to connect to or execute statements against target data.

Principal public types: TDPHSQLAnalysis.

```
class procedure Enrich(AScan: TDPHScan; AGraph: TDPHSemanticGraph);
```

25.9. Analysis.Symbols

Source file: DPH.Analysis.Symbols.pas. Builds and binds supported semantic references, including validated routine-summary reuse. Cache changes require full/incremental agreement.

Principal public types: TDPHRoutineSummary, TDPHSemanticAnalyzer.

```
constructor Create;
destructor Destroy;
function Analyze(AScan: TDPHScan; AIncremental: Boolean = True): TDPHSemanticGraph;
```

25.10. Handoff.Dossier

Source file: DPH.Handoff.Dossier.pas. Renders the selected capture as a four-part, source-linked maintenance handoff. Distinguish extracted facts, author statements, reading guidance and unresolved questions.

Principal public types: TDPHHandoffDossier.

```
class function Render(AFacts: TJSONObject; APage: Integer; AExport: Boolean = False): string;
```

25.11. Knowledge.ActionPurpose

Source file: DPH.Knowledge.ActionPurpose.pas. Validates stable action-purpose identity and twelve business explanation fields. Source targets and human intent remain separate.

```
class procedure Validate(ARecord:TJSONObject);
class function Body(ARecord:TJSONObject):string;
class function Title(ARecord:TJSONObject):string;
class function Identity(const ATarget:string):string;
```

25.12. Knowledge.ComponentDependency

Source file: DPH.Knowledge.ComponentDependency.pas. Validates attributed dependency descriptions and canonical presentation. Names do not prove deployment or licensing status.

```
class procedure Validate(ARecord:TJSONObject);
class function Body(ARecord:TJSONObject):string;
class function Title(ARecord:TJSONObject):string;
```

25.13. Knowledge.Glossary

Source file: DPH.Knowledge.Glossary.pas. Validates business terms and aliases and produces canonical text. Normalized naming rules prevent conflicting current vocabulary.

Principal public types: TDPHGlossary.

```
class function NameKey(const AName: string): string;
class procedure Validate(ARecord: TJSONObject);
class function Names(ARecord: TJSONObject): TArray<string>;
class function Body(ARecord: TJSONObject): string;
```

25.14. Knowledge.ProjectProfile

Source file: DPH.Knowledge.ProjectProfile.pas. Validates and renders an attributed application profile. Purpose, support, safe practice and unknowns support orientation.

Principal public types: TDPHProjectProfile.

```
class procedure Validate(ARecord: TJSONObject);
class function Body(ARecord: TJSONObject): string;
```

25.15. Learning.Discovery

Source file: DPH.Learning.Discovery.pas. Finds project-local documents, logs, saved screens, startup forms and referenced public websites. Retain source paths, bounded scanning, original-language text and explicit extraction status.

Principal public types: TDPHLearningDiscovery.

```
class procedure AddEvidence(AFacts, AWorkspace: TJSONObject; AAllowInternet: Boolean);
```

25.16. Parse.Forms

Source file: DPH.Parse.Forms.pas. Reads supported Delphi form definitions into component and property structure. Captured designs remain distinct from actual screenshots.

Principal public types: TDPHFormParser.

```
function Parse(ASource: TDPHSource; ADiagnostics: TDPHDiagnostics): TDPHParseResult;
```

25.17. Parse.Lexer

Source file: DPH.Parse.Lexer.pas. Converts acquired source text into ranged tokens. Malformed input and token limits must remain qualified rather than silently normalized.

Principal public types: TDPHLexer.

```
class function Tokenize(ASource: TDPHSource; ADiagnostics: TDPHDiagnostics): TDPHTokens;
```

25.18. Parse.Model

Source file: DPH.Parse.Model.pas. Defines tokens, ranges, diagnostics and intermediate syntax records. Schema/version and acquisition bounds are contracts for every downstream layer.

Principal public types: TDPHRange, TDPHDiagnostic, TDPHDiagnostics, TDPHActive, TDPHTokenKind, TDPHToken, TDPHIRNode, TDPHParseResult.

```
procedure Report(const ACode, AMessage: string; const ARange: TDPHRange; const ASeverity:
string = 'warning');
constructor Create;
destructor Destroy;
function AsJSON: TJSONObject;
function Clone: TDPHParseResult;
```

25.19. Parse.Pascal

Source file: DPH.Parse.Pascal.pas. Builds supported Pascal declarations and statement IR. Inspect parent indexes and source ranges when extending syntax coverage.

Principal public types: TDPHPascalParser.

```
function Parse(const ATokens: TDPHTokens; ADiagnostics: TDPHDiagnostics): TDPHParseResult;
```

25.20. Parse.Preprocessor

Source file: DPH.Parse.Preprocessor.pas. Evaluates supported directives and three-valued conditional activity, including allowed include resolution. Unknown activity must survive projection.

Principal public types: TDPHCondition, TDPHIncludeResolver, TDPHPreprocessor.

```
class function Evaluate(const AExpression: string; ASymbols: TDictionary<string,string>):
TDPHActive;
class function Both(A,B: TDPHActive): TDPHActive;
class function Either(A,B: TDPHActive): TDPHActive;
class function Opposite(A: TDPHActive): TDPHActive;
constructor Create(const ADefines: TArray<string>; ADiagnostics: TDPHDiagnostics; AResolver:
TDPHIncludeResolver);
destructor Destroy;
function Run(ASource: TDPHSource): TDPHTokens;
```

25.21. Parse.Resources

Source file: DPH.Parse.Resources.pas. Reads supported resource definitions and preserves qualified evidence for unsupported material. It does not establish runtime resource behavior.

Principal public types: TDPHResourceParser.

```
class function Parse(ASource: TDPHSource; ADiagnostics: TDPHDiagnostics): TDPHParseResult;
```

25.22. Project.Analysis

Source file: DPH.Project.Analysis.pas. Coordinates syntax scans, incremental reuse, context identity and generation staging. Owns acquired inventory and parse-result lifetimes.

Principal public types: TDPHScan, TDPHAnalyzer.

```
constructor Create(AInventory: TDPHInventory);
destructor Destroy;
function AsJSON(AIncludeComments: Boolean = False): TJSONObject;
function ContextHash: string;
function Stage(AStore: TDPHKnowledgeStore; AConfiguration: Int64; const AAnalyzerVersion:
string = DPHAnalyzerVersion): Int64;
function Publish(AStore: TDPHKnowledgeStore; AGeneration: Int64; AAdditionalComplete: Boolean
```

```
= True): Boolean;
constructor Create;
destructor Destroy;
function Scan(const AEntry,AConfig,APlatform: string; const ADefines,AExternalRoots:
TArray<string>; AIncremental: Boolean = True; ACodePage: Integer = 0): TDPHScan;
```

25.23. Project.Inventory

Source file: DPH.Project.Inventory.pas. Discovers the selected project and permitted dependencies under its configuration. Maintains source availability, defines and path boundaries.

Principal public types: TDPHDefinitionReference, TDPHInventory.

```
constructor Create(const AEntry,AConfig,APlatform: string; const ADefines,AExternalRoots:
TArray<string>; ACodePage: Integer = 0);
destructor Destroy;
function Capture(const AFrom,AName: string): TDPHSource;
function Manifest: TDPHManifest;
function CurrentManifest: TDPHManifest;
function Revalidate: Boolean;
```

25.24. Project.Source

Source file: DPH.Project.Source.pas. Owns original bytes, decoding, character to byte ranges and verified acquisition. Preserve byte identity and ranges when changing encodings.

Principal public types: TDPHSource.

```
constructor Create(const APath, ARelativePath: string; ADiagnostics: TDPHDiagnostics;
ACodePage: Integer = 0; ARaw: Boolean = False);
function Range(AStartChar, AEndChar: Integer): TDPHRange;
```

25.25. Publish.BrowserPolicy

Source file: DPH.Publish.BrowserPolicy.pas. Constrains local embedded navigation and internal routes. Rendering success does not replace URL containment tests.

```
class function Allowed(const AURL,ADirectory: string; AObjectPages: Boolean = False;
AHashedPages: Boolean = False): Boolean;
```

25.26. Publish.Export

Source file: DPH.Publish.Export.pas. Writes fresh offline editions and validates checksums, provenance, ranges and indexes. Learner and technical artifacts share the scoped model.

Principal public types: TDPHPublisher.

```
class procedure Export(AModel: TDPHDocumentModel; AStore: TDPHKnowledgeStore; const
ADirectory: string; AIncludeAgentInstructions: Boolean = False; AIncludeSourceNavigation:
Boolean = False; const ALocale: string = '');
class procedure Validate(const ADirectory,AExpectedFingerprint: string; ARejectStale: Boolean
= True);
class procedure WriteGuide(AModel: TDPHDocumentModel; const APath,ATitle: string; AUser:
Boolean);
```

25.27. Publish.Model

Source file: DPH.Publish.Model.pas. Builds the scoped canonical document, audiences, provenance and redacted knowledge. This is the privacy boundary shared by output formats.

Principal public types: TDPHDocumentModel.

```
constructor Create(AView: TDPHWorkspaceView; AKnowledge: TJSONArray; const ASeeds,AExclusions:
 TArray<string>; ADetached,AStale: Boolean; AImages: TJSONArray = nil; ALocalDisplay: Boolean
= False; const AAudience: string = 'all');
constructor CreateLocalizedSections(ABase: TDPHDocumentModel; const ALocale: string; ARows:
TJSONArray);
destructor Destroy;
class function Redact(const AText: string): string;
class function Scrub(AValue: TJSONValue): TJSONValue;
class function HTML(const AText: string): string;
```

25.28. Publish.OfflinePreview

Source file: DPH.Publish.OfflinePreview.pas. Serves the local preview and bundled German translation assets over loopback. It must not expose the selected project to an external network interface.

Principal public types: TDPHOfflinePreview.

```
constructor Create(const APagesDirectory, AAssetsDirectory: string);
destructor Destroy;
function URLForPage(const APagePath: string): string;
function ScriptURL: string;
function Allows(const AURL, APageDirectory: string): Boolean;
```

25.29. Publish.Translation

Source file: DPH.Publish.Translation.pas. Adapts pinned translation capabilities and revision/locale cache state. Provider configuration and named review remain separate.

Principal public types: TDPHTranslationSegment, TDPHAtlasAdapter, TDPHTranslationCache.

```
function ID: string;
function Version: string;
function External: Boolean;
function TranslateProse(const AText,ASourceLocale,ATargetLocale: string): string;
function ResolvedTargetLocale: string;
constructor Create(const APackFile: string = '');
destructor Destroy;
function ID: string;
function Version: string;
function External: Boolean;
function TranslateProse(const AText,ASourceLocale,ATargetLocale: string): string;
function ResolvedTargetLocale: string;
constructor Create(AStore: TDPHKnowledgeStore = nil);
destructor Destroy;
class function SegmentsForText(const AText: string): TArray<TDPHTranslationSegment>;
function Translate(ARevision: Int64; const ASourceLocale,ATargetLocale: string; const
ASegments: TArray<TDPHTranslationSegment>; AProvider: IDPHTranslationProvider;
AExternalConsent: Boolean = False; const ATerminologyJSON: string = '{}'): TJSONObject;
```

```
procedure Review(const AKey,AReviewer: string);
```

25.30. Storage.Backup

Source file: DPH.Storage.Backup.pas. Creates and restores consistent database plus managed-artifact copies with hash checks. Fresh destinations and outside-source paths are required.

Principal public types: TDPHProjectBackup.

```
class procedure Save(AStore: TDPHKnowledgeStore; const ADestination: string);
class procedure Restore(const ABackup,ADestination,AMigrations: string);
```

25.31. Storage.Database

Source file: DPH.Storage.Database.pas. Owns the SQLite writer, migrations, generations, knowledge revisions, artifacts and indexes. Thread ownership and transactional boundaries are mandatory.

Principal public types: TDPHKnowledgeStore.

```
procedure CheckOutsideSource(const APath: string);
constructor Create(const ADatabasePath, ASourceRoot, AMigrationDirectory: string; AUseWAL:
Boolean = False);
destructor Destroy;
class function ManifestHash(const AFiles: TDPHManifest): string;
procedure QueueWrite(const AWrite: TProc<TDPHKnowledgeStore>);
procedure ProcessQueuedWrites;
procedure InitializeProject(const AName, AProjectKind: string);
function AddConfiguration(const APlatform, ACompiler, ADefinesJSON, ASearchPathsJSON,
AEnvironment: string): Int64;
procedure SelectConfiguration(AConfigurationID: Int64);
procedure ObserveManifest(const AFiles: TDPHManifest);
function BeginGeneration(AConfigurationID: Int64; const AAnalyzerVersion: string; const
AFiles: TDPHManifest): Int64;
function AddSourceEntity(AGenerationID: Int64; const APath, AKey, AKind, AName, ASignature:
string; AStartOffset, AEndOffset: Int64; const AStructuralJSON: string = '{}'): Int64;
procedure AddDiagnostic(AGenerationID: Int64; const ASeverity,ACode,AMessage,ARangeJSON:
string);
function AddEvidence(AGenerationID: Int64; const APath, ARangeJSON, AKind, APayload,
AVerification: string): Int64;
procedure AddRelationship(AGenerationID, AFromEntityID, AToEntityID, AEvidenceID: Int64; const
 AKind, AResolution, ACondition: string);
function AddFlowNode(AGenerationID,ARoutineEntityID: Int64; const
AKind,ARangeJSON,AExpression: string): Int64;
procedure AddFlowEdge(AGenerationID,AFromNode,AToNode: Int64; const AKind,AResolution:
string);
function PublishGeneration(AGenerationID: Int64; const ARevalidatedFiles: TDPHManifest; const
ACompletenessJSON: string; AAnalysisComplete: Boolean): Boolean;
procedure FailGeneration(AGenerationID: Int64; const AReason: string);
procedure RequestReassessment;
function AddManualNote(const ATitle, ABody, AEditor: string; AEntityID: Int64 = 0): Int64;
function Status: TDPHProjectStatus;
procedure BackupTo(const ADestination: string);
function IntegrityCheck: string;
function SQLiteVersion: string;
function SupportsFTS5: Boolean;
```

```
class function ProjectRoot(const ADatabasePath: string): string;
procedure SaveWorkspaceView(AGenerationID: Int64; const AJSON: string);
function LoadWorkspaceView(AGenerationID: Int64 = 0; ASkipGenerationParse: Boolean = False):
string;
procedure RelocateSource(const ANewRoot: string; AGenerationID: Int64; const
AVerifiedManifest: TDPHManifest);
procedure BeginFacts(AGenerationID: Int64);
procedure CommitFacts;
procedure RollbackFacts;
function SaveKnowledge(AKnowledgeID,AExpectedRevision,AGenerationID: Int64; const
ATitle,ABody,AEditor,AKind,AEntityKey: string; const ADocumentKind: string = ''; const
AAudience: string = ''; const ALinksJSON: string = ''; const AQuestionJSON: string = ''; const
 AStructuredJSON: string = ''): Int64;
procedure ReviewKnowledge(AKnowledgeID,AExpectedRevision,AGenerationID: Int64; const
AReviewer,AState: string);
function KnowledgeJSON: TJSONArray;
function LoadTranslation(const AKey: string): TJSONObject;
function TranslationEntries(ARevision: Int64): TJSONArray;
procedure SaveTranslation(AEntry: TJSONObject);
procedure ReviewTranslation(const AKey,AReviewer: string);
function LinkCandidates(AGeneration: Int64; const AKind: string; const AQuery: string = ''):
TJSONArray;
function ReviewTasks(AGeneration: Int64): TJSONArray;
procedure RecordExport(AGenerationID: Int64; const ADirectory,ACanonicalHash,AScope: string;
AStale: Boolean);
function GenerationIsStale(AGenerationID: Int64): Boolean;
procedure ImportScreenshot(AGenerationID: Int64; const AFile,AAuthor,ACaption,AEntityKey:
string; const AImageKind:string='Screenshot');
function ScreenshotsJSON: TJSONArray;
function ImportDataQualityReport(AGenerationID:Int64;const
AFile,AAuthor,AContext:string):Int64;
procedure ImportObservation(AGenerationID: Int64; const AFile,AAuthor,AContext,AEntityKey:
string);
function ObservationsJSON: TJSONArray;
function RetentionReport(ARecent: Integer): TJSONArray;
function RecentRetentionCount: Integer;
procedure ApplyRetention(ARecent: Integer);
procedure RetainGeneration(AGeneration: Int64; const ALabel,AReviewer: string);
function ContinuityCandidates(AGeneration: Int64; const AQuery: string): TJSONArray;
procedure ApproveContinuity(AKnowledge,AExpectedRevision,AGeneration: Int64; const
AOldKey,ANewKey,AReviewer: string);
function ManagedArtifactPaths: TArray<string>;
procedure BuildSearchIndex(AGeneration: Int64);
function SearchProject(AGeneration: Int64; const AQuery,AKind: string; AOffset: Integer):
TJSONArray;
```

25.32. Storage.Types

Source file: DPH.Storage.Types.pas. Defines persistent status and manifest types shared by analysis and storage. Changes require serialization and lifecycle compatibility checks.

Principal public types: TDPHManifestFile, TDPHProjectStatus.

This unit exposes shared data types/constants; inspect its declarations for the exact fields and current version values.

25.33. UI.ActionPurpose

Source file: DPH.UI.ActionPurpose.pas. Collects an action explanation in a serialized dialog. Keep user purpose and supporting captured target clearly identified.

Principal public types: TDPHActionPurposeForm.

```
procedure ApplyProfileClick(Sender: TObject);
procedure LoadRecord(ARecord: TJSONObject; const AAuthor: string);
function RecordJSON: TJSONObject;
```

25.34. UI.ComponentDependency

Source file: DPH.UI.ComponentDependency.pas. Collects attributed component/dependency descriptions. It is a knowledge editor, not an installer or dependency resolver.

Principal public types: TDPHComponentDependencyForm.

```
procedure ApplyProfileClick(Sender: TObject);
procedure LoadRecord(ARecord: TJSONObject; const AAuthor: string);
function RecordJSON: TJSONObject;
```

25.35. UI.DatabaseDictionary

Source file: DPH.UI.DatabaseDictionary.pas. Collects authored dictionary descriptions. This dialog does not become a live database administration engine.

Principal public types: TDPHDatabaseDictionaryForm.

```
procedure ApplyProfileClick(Sender: TObject);
procedure LoadRecord(ARecord: TJSONObject; const AAuthor: string);
function RecordJSON: TJSONObject;
```

25.36. UI.Glossary

Source file: DPH.UI.Glossary.pas. Collects a business term and its supporting meaning/aliases. Validate before applying the draft to saved knowledge.

Principal public types: TDPHGlossaryForm.

```
procedure ApplyTermClick(Sender: TObject);
procedure LoadRecord(ARecord: TJSONObject; const AAuthor: string);
function RecordJSON: TJSONObject;
```

25.37. UI.Main

Source file: DPH.UI.Main.pas. Binds serialized FireMonkey controls to worker requests and owned models. Preserve pending drafts, busy state, selection identity and close guards.

Principal public types: TDPHMainForm.

```
procedure KeyboardFilterChange(Sender: TObject);
procedure ActionPurposeFilterChange(Sender:TObject);
```

```
procedure ActionPurposeSelectionClick(Sender:TObject);
procedure ActionPurposeSourceClick(Sender:TObject);
procedure ActionPurposeExplainClick(Sender:TObject);
procedure ActionPurposeKnowledgeClick(Sender:TObject);
procedure ActionPurposeEditClick(Sender:TObject);
procedure InheritanceFilterChange(Sender:TObject);
procedure InheritanceSelectionClick(Sender:TObject);
procedure InheritanceOriginChange(Sender:TObject);
procedure InheritanceSourceClick(Sender:TObject);
procedure DependencyFilterChange(Sender:TObject);
procedure DependencySelectionClick(Sender:TObject);
procedure DependencySourceClick(Sender:TObject);
procedure DependencyClassClick(Sender:TObject);
procedure DependencyEditClick(Sender:TObject);
procedure DependencyKnowledgeClick(Sender:TObject);
procedure LayoutFilterChange(Sender:TObject);
procedure LayoutSelectionClick(Sender:TObject);
procedure LayoutApplyClick(Sender:TObject);
procedure LayoutSourceClick(Sender:TObject);
procedure DynamicFilterChange(Sender:TObject);
procedure DynamicSelectionClick(Sender:TObject);
procedure DynamicReferenceChange(Sender:TObject);
procedure DynamicExplainClick(Sender:TObject);
procedure CommandsFilterChange(Sender: TObject);
procedure CommandsSelectionClick(Sender: TObject);
procedure CommandsSourceClick(Sender: TObject);
procedure CommandsHandlerChange(Sender: TObject);
procedure CommandsHandlerClick(Sender: TObject);
procedure KeyboardSelectionClick(Sender: TObject);
procedure KeyboardSourceClick(Sender: TObject);
procedure ComparisonClick(Sender: TObject);
procedure ProjectMemberSelectionClick(Sender: TObject);
procedure ComparisonSelectionClick(Sender: TObject);
procedure QualityClick(Sender: TObject);
procedure QualitySelectionClick(Sender: TObject);
procedure TranslationClick(Sender: TObject);
procedure ProjectProfileClick(Sender: TObject);
procedure TranslationSelectionClick(Sender: TObject);
procedure ValidateBundleClick(Sender: TObject);
procedure QuestionsClick(Sender: TObject);
procedure QuestionSelectionClick(Sender: TObject);
procedure ObservationsClick(Sender: TObject);
procedure ObservationSelectionClick(Sender: TObject);
procedure ReviewTasksClick(Sender: TObject);
procedure ReviewTaskSelectionClick(Sender: TObject);
procedure KnowledgeLinksClick(Sender: TObject);
procedure OpenKnowledgeLinkClick(Sender: TObject);
procedure CoverageInspectClick(Sender: TObject);
procedure WorkspacePageChange(Sender: TObject);
procedure LearnerTabClick(Sender: TObject);
procedure WholeProgramGraphClick(Sender: TObject);
procedure AllFormsGraphClick(Sender: TObject);
procedure AllDataGraphClick(Sender: TObject);
```

```
procedure ProjectSearchClick(Sender: TObject);
procedure ProjectSearchSelectionClick(Sender: TObject);
procedure RetentionClick(Sender: TObject);
procedure DatabaseToolsClick(Sender: TObject);
procedure ContinuityClick(Sender: TObject);
procedure ContinuitySelectionClick(Sender: TObject);
procedure OpenKnownProjectClick(Sender: TObject);
procedure ForgetProjectClick(Sender: TObject);
procedure RefreshProjectsClick(Sender: TObject);
procedure HelpClick(Sender: TObject);
procedure ImportScreenshotClick(Sender: TObject);
procedure ScreenshotClick(Sender: TObject);
procedure ScreenshotPoint(Sender:TObject;Button:TMouseButton;Shift:TShiftState;X,Y:Single);
procedure ScreenshotMarkerPaint(Sender:TObject;Canvas:TCanvas);
procedure ScreenshotCalloutClick(Sender:TObject);
procedure ScreenshotViewChange(Sender: TObject);
procedure ScreenshotInspectClick(Sender: TObject);
procedure ScreenshotReturnClick(Sender: TObject);
procedure DatabaseERFilterChange(Sender:TObject);
procedure DatabaseERFocusClick(Sender:TObject);
procedure DatabaseERSelectionChange(Sender:TObject);
procedure DatabaseEROpenClick(Sender:TObject);
procedure DatabaseERResize(Sender:TObject);
procedure DatabaseERPaintPaint(Sender:TObject;Canvas:TCanvas);
procedure
DatabaseERMouseDown(Sender:TObject;Button:TMouseButton;Shift:TShiftState;X,Y:Single);
procedure DatabaseDictionaryEditClick(Sender:TObject);
procedure DatabaseDictionaryKnowledgeClick(Sender:TObject);
procedure FileFormatsFilterChange(Sender:TObject);
procedure FileFormatsSelectionChange(Sender:TObject);
procedure FileFormatsSourceClick(Sender:TObject);
procedure FileFormatsExplainClick(Sender:TObject);
procedure DataRetentionFilterChange(Sender:TObject);
procedure DataRetentionSelectionChange(Sender:TObject);
procedure DataRetentionNewClick(Sender:TObject);
procedure DataRetentionOpenClick(Sender:TObject);
procedure DataQualityImportClick(Sender:TObject);
procedure TargetRecoveryFilterChange(Sender:TObject);
procedure TargetRecoverySelectionChange(Sender:TObject);
procedure TargetRecoveryNewClick(Sender:TObject);
procedure TargetRecoveryOpenClick(Sender:TObject);
procedure BusinessProcessFilterChange(Sender:TObject);
procedure BusinessProcessSelectionChange(Sender:TObject);
procedure BusinessProcessNewClick(Sender:TObject);
procedure BusinessProcessOpenClick(Sender:TObject);
procedure CalculationEvidenceFilterChange(Sender:TObject);
procedure CalculationEvidenceSelectionChange(Sender:TObject);
procedure CalculationEvidenceSourceClick(Sender:TObject);
procedure CalculationEvidenceExplainClick(Sender:TObject);
procedure CalculationEvidenceDefineClick(Sender:TObject);
procedure CalculationEvidenceOpenClick(Sender:TObject);
procedure CalculationFilterChange(Sender:TObject);
procedure CalculationSelectionChange(Sender:TObject);
```

```
procedure CalculationNewClick(Sender:TObject);
procedure CalculationOpenClick(Sender:TObject);
procedure CalculationEditClick(Sender:TObject);
procedure CalculationApplyClick(Sender:TObject);
procedure BusinessRuleFilterChange(Sender:TObject);
procedure BusinessRuleSelectionChange(Sender:TObject);
procedure BusinessRuleNewClick(Sender:TObject);
procedure BusinessRuleOpenClick(Sender:TObject);
procedure BusinessRuleEditClick(Sender:TObject);
procedure BusinessRuleApplyClick(Sender:TObject);
procedure DepartmentalWorkflowFilterChange(Sender:TObject);
procedure DepartmentalWorkflowSelectionChange(Sender:TObject);
procedure DepartmentalWorkflowDiagramClick(Sender:TObject);
procedure DepartmentalWorkflowNewClick(Sender:TObject);
procedure DepartmentalWorkflowOpenClick(Sender:TObject);
procedure DepartmentalWorkflowEditClick(Sender:TObject);
procedure WorkflowApplyClick(Sender:TObject);
procedure StructuredEditorChange(Sender:TObject);
procedure WorkflowStepClick(Sender:TObject);
procedure WorkflowStepChange(Sender:TObject);
procedure WorkflowStepAction(Sender:TObject);
procedure RefreshWorkflowSteps;
procedure BusinessProcessEditClick(Sender:TObject);
procedure BusinessProcessApplyClick(Sender:TObject);
procedure DataQualityFilterChange(Sender:TObject);
procedure DataQualitySelectionChange(Sender:TObject);
procedure DataQualityNewClick(Sender:TObject);
procedure DataQualityOpenClick(Sender:TObject);
procedure FileFormatsContractClick(Sender:TObject);
procedure DatabaseSchemaFilterChange(Sender:TObject);
procedure DatabaseSchemaSelectionChange(Sender:TObject);
procedure DatabaseSchemaSourceClick(Sender:TObject);
procedure DatabaseSchemaFollowClick(Sender:TObject);
procedure DatabaseInventoryFilterChange(Sender:TObject);
procedure DatabaseInventorySelectionChange(Sender:TObject);
procedure DatabaseInventoryFollowClick(Sender:TObject);
procedure DatabaseInventoryTraceClick(Sender:TObject);
procedure DatabaseInventorySourceClick(Sender:TObject);
procedure HistorySelectionChange(Sender:TObject);
procedure HistoryCompareClick(Sender:TObject);
procedure HistoryModeChange(Sender:TObject);
procedure HistoryFilterChange(Sender:TObject);
procedure HistoryRowClick(Sender:TObject);
procedure HistoryPaint(Sender:TObject;Canvas:TCanvas);
procedure LearningClick(Sender: TObject);
procedure LearningShouldLoad(Sender: TObject; const AURL: string; var ACancel: Boolean);
procedure RelocateClick(Sender: TObject);
procedure TaskFilterChange(Sender: TObject);
procedure TaskProgressClick(Sender: TObject);
procedure CardFilterChange(Sender:TObject);
procedure CardSelectionChange(Sender:TObject);
procedure CardEditClick(Sender:TObject);
procedure CardSaveClick(Sender:TObject);
```

```
procedure CardCancelClick(Sender:TObject);
procedure CardChanged(Sender:TObject);
procedure CardRelatedClick(Sender:TObject);
procedure TaskSelectionChange(Sender: TObject);
procedure TaskNavigate(Sender: TObject);
procedure TaskEditClick(Sender: TObject);
procedure TaskOpenNotes(Sender: TObject);
procedure TaskEditorChanged(Sender: TObject);
procedure TutorStepSelection(Sender: TObject);
procedure TutorStepChanged(Sender: TObject);
procedure TutorStepAction(Sender: TObject);
procedure TaskSaveClick(Sender: TObject);
procedure TaskCancelClick(Sender: TObject);
procedure SetupClick(Sender: TObject);
procedure FormCreate(Sender: TObject);
procedure FormDestroy(Sender: TObject);
procedure FormCloseQuery(Sender: TObject; var CanClose: Boolean);
procedure OpenProjectClick(Sender: TObject);
procedure OpenCacheClick(Sender: TObject);
procedure CloseButtonClick(Sender: TObject);
procedure AllowInternetChange(Sender: TObject);
procedure PollTimerTimer(Sender: TObject);
procedure ChangeTimerTimer(Sender: TObject);
procedure SearchChange(Sender: TObject);
procedure ExplorerClick(Sender: TObject);
procedure ExplorerExpandClick(Sender: TObject);
procedure DetailsClick(Sender: TObject);
procedure DetailsSourceClick(Sender: TObject);
procedure DetailsObjectClick(Sender: TObject);
procedure RelationsClick(Sender: TObject);
procedure BackClick(Sender: TObject);
procedure ParentClick(Sender: TObject);
procedure NextPageClick(Sender: TObject);
procedure BookmarkClick(Sender: TObject);
procedure BookmarksClick(Sender: TObject);
procedure ViewChange(Sender: TObject);
procedure GraphPaintPaint(Sender: TObject; Canvas: TCanvas);
procedure GraphMouseDown(Sender: TObject; Button: TMouseButton; Shift: TShiftState; X,Y:
Single);
procedure ScreenPaintPaint(Sender: TObject; Canvas: TCanvas);
procedure ScreenMouseDown(Sender: TObject; Button: TMouseButton; Shift: TShiftState; X,Y:
Single);
procedure FullscreenClick(Sender: TObject);
procedure FormKeyDown(Sender: TObject; var Key: Word; var KeyChar: Char; Shift: TShiftState);
procedure FormResize(Sender: TObject);
procedure KnowledgeClick(Sender: TObject);
procedure NewKnowledgeClick(Sender: TObject);
procedure SaveKnowledgeClick(Sender: TObject);
procedure GlossaryFilterChange(Sender: TObject);
procedure GlossaryClick(Sender: TObject);
procedure EditGlossaryClick(Sender: TObject);
procedure OpenGlossaryKnowledgeClick(Sender: TObject);
procedure RefreshKnowledgeClick(Sender: TObject);
```

```
procedure ReviewKnowledgeClick(Sender: TObject);
procedure KnowledgeFilterChange(Sender: TObject);
procedure KnowledgeEdited(Sender: TObject);
procedure DiscardKnowledgeClick(Sender: TObject);
procedure PublishClick(Sender: TObject);
procedure PreviewClick(Sender: TObject);
procedure BrowserShouldLoad(Sender: TObject; const AURL: string; var ACancel: Boolean);
procedure LanguageChanged(Sender: TObject; const AOldLanguage, ANewLanguage: string; const
AGeneration: Cardinal);
procedure OpenSelection(const AEntry,ADatabase: string; AUnsaved: Boolean = False);
procedure LoadView(ADocument: TJSONObject; AScreenshots: TJSONArray = nil; ADeferLearning:
Boolean = False);
procedure LoadKnowledge(ARows: TJSONArray; ADeferInitialViews: Boolean = False);
procedure LoadScreenshots(ARows: TJSONArray);
procedure LoadReviewTasks(ARows: TJSONArray);
procedure LoadComparison(AReport: TJSONObject);
procedure LoadScreenHistory(AReport:TJSONObject;const
ABeforePNG,AAfterPNG,ADifferencePNG:TBytes);
procedure LoadTranslations(ARows: TJSONArray);
```

25.38. UI.ProjectProfile

Source file: DPH.UI.ProjectProfile.pas. Collects application-profile fields in an IDE-editable dialog. Applying returns a draft; general knowledge saving persists it.

Principal public types: TDPHProjectProfileForm.

```
procedure ApplyProfileClick(Sender: TObject);
procedure LoadRecord(ARecord: TJSONObject; const AAuthor: string);
function RecordJSON: TJSONObject;
```

25.39. Workspace.ActionPurposes

Source file: DPH.Workspace.ActionPurposes.pas. Projects attributed action-purpose records with supported source links. Business intent must remain supplied knowledge.

```
class function
Inspect(AView:TDPHWorkspaceView;AComponents:TJSONObject;AKnowledge:TJSONArray):TJSONObject;
class function Describe(ARow:TJSONObject;AKnowledge:TJSONArray):string;
class function FindRecord(AKnowledge:TJSONArray;const AIdentity:string):TJSONObject;
class function State(ARow:TJSONObject;AKnowledge:TJSONArray):string;
```

25.40. Workspace.Catalog

Source file: DPH.Workspace.Catalog.pas. Stores local database navigation and revision-specific learner progress. Atomic replacement protects the existing preference file.

Principal public types: TDPHWorkspaceCatalog.

```
constructor Create(const APath: string);
destructor Destroy;
procedure Remember(const ADatabase: string);
procedure Forget(const ADatabase: string);
procedure Save;
function Projects: TJSONArray;
```

```
function LearningProgress(const ADatabase, ALearner, ALesson: string; ARevision: Int64):
Integer;
procedure SetLearningProgress(const ADatabase, ALearner, ALesson: string; ARevision: Int64;
ACheckedSteps: Integer);
class function DefaultPath: string;
```

25.41. Workspace.Commands

Source file: DPH.Workspace.Commands.pas. Projects commands, actions and handler connections. Keep command declarations separate from observed invocation.

Principal public types: TDPHCommandInventory.

```
class function Inspect(AView: TDPHWorkspaceView; AKeyboard: TJSONObject): TJSONObject;
class function Describe(ARow: TJSONObject): string;
```

25.42. Workspace.Comparison

Source file: DPH.Workspace.Comparison.pas. Compares saved captured evidence between generations. It is not reconstruction of unavailable historical source text.

Principal public types: TDPHRevisionComparison.

```
class function Compare(ABefore,AAfter: TJSONObject; const AFilter: string = ''; AOffset:
Integer = 0): TJSONObject;
class function EvidenceText(ARow: TJSONObject): string;
```

25.43. Workspace.ComponentDependencies

Source file: DPH.Workspace.ComponentDependencies.pas. Projects component dependency evidence for focused inspection. A source dependency is not proof of deployed runtime availability.

```
class function Inspect(AView:TDPHWorkspaceView;AComponents:TJSONObject):TJSONObject;
class function Describe(ARow:TJSONObject;AKnowledge:TJSONArray):string;
class function FindRecord(AKnowledge:TJSONArray;const AClass:string):TJSONObject;
```

25.44. Workspace.DatabaseConnections

Source file: DPH.Workspace.DatabaseConnections.pas. Projects source-derived database connection declarations. Never interpret a declaration as successful connectivity.

Principal public types: TDPHDatabaseConnections.

```
class function Inspect(AView:TDPHWorkspaceView):TJSONObject;
class function Describe(ARow:TJSONObject):string;
class function Lineage(AView:TDPHWorkspaceView;AInventory:TJSONObject;const
ASeed,ADirection:string;ALimit:Integer=200):TJSONObject;
class function DescribeLineage(ATrace:TJSONObject):string;
```

25.45. Workspace.DatabaseSchema

Source file: DPH.Workspace.DatabaseSchema.pas. Projects supported schema definitions and supplied descriptions. It does not introspect a live target database automatically.

```
class function NewDictionary(ARow:TJSONObject):TJSONObject;
class procedure ValidateDictionary(ARecord:TJSONObject);
```

```
class function DictionaryBody(ARecord:TJSONObject):string;
class function DictionaryTitle(ARecord:TJSONObject):string;
class function
FindDictionary(AKnowledge:TJSONArray;ARow:TJSONObject;AHistorical:Boolean=False):TJSONObject;
class function DictionaryIdentity(const ASource,AHash,AID:string):string;
class function Inspect(AView:TDPHWorkspaceView):TJSONObject;
class function Describe(ARow:TJSONObject):string;
class function Diagram(AInventory:TJSONObject;const
AFilter,AFocusID:string;AOffset,ALimit,AMaxColumns:Integer):TJSONObject;
```

25.46. Workspace.DynamicControls

Source file: DPH.Workspace.DynamicControls.pas. Projects supported dynamic-control patterns. Source evidence cannot guarantee all runtime-created controls are reconstructed.

Principal public types: TDPHDynamicControls.

```
class function Inspect(AView:TDPHWorkspaceView):TJSONObject;
class function Describe(ARow:TJSONObject):string;
```

25.47. Workspace.FormInheritance

Source file: DPH.Workspace.FormInheritance.pas. Projects supported saved-form inheritance. Incomplete ancestor material requires explicit qualification.

Principal public types: TDPHFormInheritance.

```
class function Inspect(AView: TDPHWorkspaceView): TJSONObject;
class function Describe(ARow: TJSONObject): string;
```

25.48. Workspace.Graphs

Source file: DPH.Workspace.Graphs.pas. Projects bounded graph views for investigation. Preserve source identity, edge qualification and selection routes.

Principal public types: TDPHGraphProjection.

```
class function Workflow(ARow:TJSONObject; AKnowledge:TJSONArray;
AGeneration:Int64):TJSONObject;
class function WorkflowSVG(AGraph:TJSONObject):string;
class function Group(AView: TDPHWorkspaceView; AGraph: TJSONObject; const ALevel: string;
ALimit: Integer = 120): TJSONObject;
```

25.49. Workspace.Keyboard

Source file: DPH.Workspace.Keyboard.pas. Projects keyboard/shortcut evidence and verified source navigation. Preserve detached-source limits and original ranges.

Principal public types: TDPHKeyboardInspector.

```
class function Inspect(AView: TDPHWorkspaceView): TJSONObject;
class function Describe(ARow: TJSONObject): string;
class function ReadSource(AView: TDPHWorkspaceView; const AEntry: string; ARow,ARange:
TJSONObject): string;
```

25.50. Workspace.LayoutRules

Source file: DPH.Workspace.LayoutRules.pas. Explains captured alignment, anchors and spacing. Educational resize projections are not observed runtime geometry.

Principal public types: TDPHLayoutRules.

```
class function Inspect(AComponents:TJSONObject):TJSONObject;
class function Describe(ARow:TJSONObject; AWidthDelta,AHeightDelta:Integer):string;
```

25.51. Workspace.Model

Source file: DPH.Workspace.Model.pas. Owns the captured workspace view and typed teaching-record validators/canonical renderers. Keep structured metadata and readable body consistent.

Principal public types: TDPHWorkspaceView.

```
constructor Create(ADocument: TJSONObject);
destructor Destroy;
class function Capture(AScan: TDPHScan; AGraph: TDPHSemanticGraph; AGeneration: Int64; const
AEntry,AConfig,APlatform: string; const ADefines,AExternalRoots: TArray<string>): TJSONObject;
function Symbol(const AKey: string): TJSONObject;
function Neighbors(const AKey,ADirection: string): TJSONArray;
function Search(const AQuery,AKind,AScope: string; AOffset: Integer = 0; ALimit: Integer =
250): TJSONArray;
function Hierarchy(const AExpanded: TArray<string>; AOffset: Integer = 0; ALimit: Integer =
250): TJSONArray;
function SourceFile(const APath: string): TJSONObject;
function Properties(const AKey: string): TJSONArray;
function Details(const AKey: string): TJSONArray;
function Graph(const AKind,AKey: string; ALimit: Integer = 120): TJSONObject;
function FlagsText: string;
function Coverage: TJSONObject;
function Questions(AKnowledge: TJSONArray = nil): TJSONArray;
function ProjectMembers: TJSONArray;
class function NewDataRetentionPolicy:string;
class procedure ValidateDataRetentionRecord(ARecord:TJSONObject);
class function IsDataRetentionPolicy(ARow:TJSONObject):Boolean;
class function DescribeDataRetentionPolicy(ARow:TJSONObject):string;
class function NewDepartmentalWorkflow:TJSONObject;
class function NewWorkflowStep:TJSONObject;
class procedure ValidateWorkflowStep(AStep:TJSONObject);
class procedure ValidateDepartmentalWorkflow(ARecord:TJSONObject);
class function DepartmentalWorkflowBody(ARecord:TJSONObject):string;
class function IsDepartmentalWorkflow(ARow:TJSONObject):Boolean;
class function DescribeDepartmentalWorkflow(ARow:TJSONObject):string;
class function NewBusinessProcess:TJSONObject;
class procedure ValidateBusinessProcess(ARecord:TJSONObject);
class function BusinessProcessBody(ARecord:TJSONObject):string;
class function IsBusinessProcess(ARow:TJSONObject):Boolean;
class function DescribeBusinessProcess(ARow:TJSONObject):string;
class function NewCalculation:TJSONObject;
class function NewLearningLesson:TJSONObject;
class function NewLearningCard:TJSONObject;
class procedure ValidateLearningCard(ARecord:TJSONObject);
```

```
class function LearningCardBody(ARecord:TJSONObject):string;
class procedure ValidateLearningLesson(ARecord:TJSONObject);
class function LearningLessonBody(ARecord:TJSONObject):string;
class function IsLearningLesson(ARow:TJSONObject):Boolean;
class procedure ValidateCalculation(ARecord:TJSONObject);
class function CalculationBody(ARecord:TJSONObject):string;
class function IsCalculation(ARow:TJSONObject):Boolean;
class function DescribeCalculation(ARow:TJSONObject):string;
function Calculations:TJSONObject;
class function DescribeCalculationEvidence(ARow:TJSONObject):string;
class function NewBusinessRule:TJSONObject;
class procedure ValidateBusinessRule(ARecord:TJSONObject);
class function BusinessRuleBody(ARecord:TJSONObject):string;
class function IsBusinessRule(ARow:TJSONObject):Boolean;
class function DescribeBusinessRule(ARow:TJSONObject):string;
class function NewTargetRecoveryPlan:string;
class procedure ValidateTargetRecoveryRecord(ARecord:TJSONObject);
class function IsTargetRecoveryPlan(ARow:TJSONObject):Boolean;
class function DescribeTargetRecoveryPlan(ARow:TJSONObject):string;
class function NewDataQualityRule:string;
class procedure ValidateDataQualityRecord(ARecord:TJSONObject);
class function IsDataQualityRule(ARow:TJSONObject):Boolean;
class function DescribeDataQualityRule(ARow:TJSONObject):string;
function FileFormats: TJSONObject;
class function DescribeFileFormat(ARow:TJSONObject):string;
class function FileFormatContractTitle(ARow:TJSONObject):string;
class function SelectionHash(const AConfig,APlatform: string; const ADefines,AExternalRoots:
TArray<string>; ACodePage: Integer = 0): string;
class function IsHandoffProject(ADocument: TJSONObject): Boolean;
```

25.52. Workspace.Screen

Source file: DPH.Workspace.Screen.pas. Projects source-based form geometry for inspection. Results teach captured layout and do not claim an observed runtime screenshot.

Principal public types: TDPHScreenLayout.

```
class function Reconstruct(AView: TDPHWorkspaceView; const AFile: string; AIncludeKeys:
Boolean = True): TJSONObject;
class function Gallery(AView: TDPHWorkspaceView): TJSONObject;
```

25.53. Workspace.ScreenHistory

Source file: DPH.Workspace.ScreenHistory.pas. Compares supplied/captured screen material with generation-qualified identity. Keep actual images and reconstructed forms distinct.

```
class function CompareImages(ABefore,AAfter:TJSONObject;out
ABeforePNG,AAfterPNG,ADifferencePNG:TBytes):TJSONObject;
class function CompareLayouts(ABefore,AAfter:TDPHWorkspaceView;const
ABeforeFile,AAfterFile:string):TJSONObject;
class function Describe(AReport:TJSONObject):string;
```

25.54. Workspace.Worker

Source file: DPH.Workspace.Worker.pas. Serializes background commands and returns owned response objects. Preserve queue bounds, COM lifecycle and response ownership transfers.

Principal public types: TDPHWorkspaceRequest, TDPHWorkspaceResponse, TDPHWorkspaceWorker.

```
destructor Destroy;
constructor Create(const ABaseDirectory: string);
destructor Destroy;
procedure Submit(const ARequest: TDPHWorkspaceRequest);
function Poll(out AResponse: TDPHWorkspaceResponse): Boolean;
procedure Stop;
```

26. Persisted schema and migration reference

26.1. How to read a table definition

The following definitions are extracted from the numbered SQL migrations. A primary key identifies a row. A unique constraint prevents duplicate identities under the listed columns. A foreign key requires the referenced project/generation/entity to exist. A check constraint restricts allowed values or valid JSON. Triggers add lifecycle or immutability protection that cannot be inferred from column names alone.

The reference separates table definitions from later alterations and the names of indexes/triggers. Consult the corresponding migration for the complete trigger body or transactional implementation. The final schema is the cumulative result of migrations 001 through 009, not merely migration 001. structured_json, revision links, search and translation cache therefore need their later additions to be understood correctly.

26.2. Migration 001 foundation

Foundation project/configuration identity, immutable captures, source facts and authored history. Source: migrations/001_foundation.sql.

```
CREATE TABLE projects (
 project_id INTEGER PRIMARY KEY CHECK(project_id=1), name TEXT NOT NULL,
 root_uri TEXT NOT NULL, project_kind TEXT NOT NULL, active_config INTEGER,
 created_at TEXT NOT NULL, last_opened_at TEXT NOT NULL,
 FOREIGN KEY(active_config,project_id) REFERENCES configurations(config_id,project_id)
);
CREATE TABLE configurations (
 config_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL REFERENCES projects(project_id),
 platform TEXT NOT NULL, compiler TEXT NOT NULL, defines_json TEXT NOT NULL,
 search_paths_json TEXT NOT NULL, environment_label TEXT NOT NULL,
 fingerprint TEXT NOT NULL CHECK(length(fingerprint)=64),
 UNIQUE(config_id,project_id), UNIQUE(project_id,fingerprint),
 CHECK(json_valid(defines_json) AND json_valid(search_paths_json))
);
CREATE TABLE analysis_generations (
 generation_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL REFERENCES
projects(project_id),
 config_id INTEGER NOT NULL, started_at TEXT NOT NULL, completed_at TEXT,
 analyzer_version TEXT NOT NULL, config_fingerprint TEXT NOT NULL,
```

```
 manifest_hash TEXT NOT NULL CHECK(length(manifest_hash)=64),
 status TEXT NOT NULL CHECK(status IN ('running','current','partial','failed','superseded')),
 previous_generation_id INTEGER, completeness_summary_json TEXT NOT NULL DEFAULT '{}'
CHECK(json_valid(completeness_summary_json)),
 UNIQUE(generation_id,project_id),
 FOREIGN KEY(config_id,project_id) REFERENCES configurations(config_id,project_id),
 FOREIGN KEY(previous_generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id)
);
CREATE TABLE project_state (
 project_id INTEGER PRIMARY KEY REFERENCES projects(project_id), published_generation_id
INTEGER,
 modification_detected INTEGER NOT NULL DEFAULT 0 CHECK(modification_detected IN (0,1)),
 analysis_stale INTEGER NOT NULL DEFAULT 1 CHECK(analysis_stale IN (0,1)),
 knowledge_review_needed INTEGER NOT NULL DEFAULT 0 CHECK(knowledge_review_needed IN (0,1)),
 analysis_failed INTEGER NOT NULL DEFAULT 0 CHECK(analysis_failed IN (0,1)),
 reassessment_requested INTEGER NOT NULL DEFAULT 0 CHECK(reassessment_requested IN (0,1)),
 FOREIGN KEY(published_generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id)
);
CREATE TABLE source_files (
 file_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL REFERENCES projects(project_id),
 canonical_relpath TEXT NOT NULL, kind TEXT NOT NULL, current_observed_hash TEXT,
 last_analyzed_hash TEXT, availability_status TEXT NOT NULL
 CHECK(availability_status IN ('available','missing','excluded','unreadable')),
 reassessment_flag INTEGER NOT NULL CHECK(reassessment_flag IN (0,1)), encoding TEXT NOT NULL,
 observed_size INTEGER NOT NULL CHECK(observed_size>=0),
 in_manifest INTEGER NOT NULL DEFAULT 1 CHECK(in_manifest IN (0,1)),
 UNIQUE(file_id,project_id), UNIQUE(project_id,canonical_relpath)
);
CREATE TABLE artifact_blobs (
 blob_id TEXT PRIMARY KEY CHECK(length(blob_id)=64), storage_ref TEXT NOT NULL,
 size_bytes INTEGER NOT NULL CHECK(size_bytes>=0)
);
CREATE TABLE file_revisions (
 file_revision_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, file_id INTEGER NOT NULL,
 sha256 TEXT NOT NULL CHECK(length(sha256)=64), size_bytes INTEGER NOT NULL
CHECK(size_bytes>=0),
 revision_time TEXT NOT NULL, source_generation_id INTEGER NOT NULL,
 snapshot_blob_id TEXT REFERENCES artifact_blobs(blob_id),
 UNIQUE(file_id,sha256), UNIQUE(file_revision_id,file_id,project_id),
 FOREIGN KEY(file_id,project_id) REFERENCES source_files(file_id,project_id),
 FOREIGN KEY(source_generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id)
);
CREATE TABLE generation_files (
 generation_id INTEGER NOT NULL, project_id INTEGER NOT NULL, file_id INTEGER NOT NULL,
 file_revision_id INTEGER, captured_hash TEXT NOT NULL,
 captured_size INTEGER NOT NULL CHECK(captured_size>=0), availability TEXT NOT NULL,
 kind TEXT NOT NULL, encoding TEXT NOT NULL,
 PRIMARY KEY(generation_id,file_id), UNIQUE(generation_id,file_revision_id),
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
```

```
 FOREIGN KEY(file_id,project_id) REFERENCES source_files(file_id,project_id),
 FOREIGN KEY(file_revision_id,file_id,project_id) REFERENCES
file_revisions(file_revision_id,file_id,project_id),
 CHECK((availability='available' AND file_revision_id IS NOT NULL AND
length(captured_hash)=64)
 OR (availability IN ('missing','excluded','unreadable') AND file_revision_id IS NULL AND
captured_hash=''))
);
CREATE TABLE entities (
 entity_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL REFERENCES projects(project_id),
 entity_kind TEXT NOT NULL, canonical_key TEXT NOT NULL, display_name TEXT NOT NULL,
 owner_file_id INTEGER, lifecycle_state TEXT NOT NULL DEFAULT 'active',
 UNIQUE(entity_id,project_id), UNIQUE(project_id,canonical_key,entity_kind),
 FOREIGN KEY(owner_file_id,project_id) REFERENCES source_files(file_id,project_id)
);
CREATE TABLE entity_versions (
 entity_version_id INTEGER PRIMARY KEY, entity_id INTEGER NOT NULL, project_id INTEGER NOT
NULL,
 generation_id INTEGER NOT NULL, signature TEXT NOT NULL, structural_json TEXT NOT NULL,
 source_revision_id INTEGER, start_line INTEGER, end_line INTEGER, start_offset INTEGER,
end_offset INTEGER,
 analysis_status TEXT NOT NULL, UNIQUE(entity_id,generation_id),
UNIQUE(entity_version_id,generation_id),
 FOREIGN KEY(entity_id,project_id) REFERENCES entities(entity_id,project_id),
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
 FOREIGN KEY(generation_id,source_revision_id) REFERENCES
generation_files(generation_id,file_revision_id),
 CHECK(start_offset IS NULL OR (start_offset>=0 AND end_offset>=start_offset)),
 CHECK(start_line IS NULL OR (start_line>=1 AND end_line>=start_line)),
CHECK(json_valid(structural_json))
);
CREATE TABLE evidence (
 evidence_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, generation_id INTEGER NOT NULL,
 file_revision_id INTEGER NOT NULL, source_range_json TEXT NOT NULL
CHECK(json_valid(source_range_json)),
 evidence_kind TEXT NOT NULL, payload_ref TEXT NOT NULL, analyzer_version TEXT NOT NULL,
 capture_time TEXT NOT NULL, verification_state TEXT NOT NULL
 CHECK(verification_state IN ('verified','inferred','needs_review','rejected')),
 UNIQUE(evidence_id,generation_id), UNIQUE(evidence_id,project_id),
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
 FOREIGN KEY(generation_id,file_revision_id) REFERENCES
generation_files(generation_id,file_revision_id)
);
CREATE TABLE relationships (
 relationship_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, generation_id INTEGER NOT
NULL,
 from_entity_id INTEGER NOT NULL, to_entity_id INTEGER, relation_kind TEXT NOT NULL,
 resolution_status TEXT NOT NULL CHECK(resolution_status IN
 ('confirmed_static','possible_dynamic','inferred','runtime_observed','unresolved')),
 evidence_id INTEGER NOT NULL, condition_variant TEXT,
 UNIQUE(relationship_id,project_id),
```

```
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
 FOREIGN KEY(from_entity_id,generation_id) REFERENCES
entity_versions(entity_id,generation_id),
 FOREIGN KEY(to_entity_id,generation_id) REFERENCES entity_versions(entity_id,generation_id),
 FOREIGN KEY(evidence_id,generation_id) REFERENCES evidence(evidence_id,generation_id)
);
CREATE TABLE diagnostics (
 diagnostic_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, generation_id INTEGER NOT
NULL,
 severity TEXT NOT NULL, code TEXT NOT NULL, message TEXT NOT NULL, source_range_json TEXT,
 affected_entity_id INTEGER, resolved_at TEXT,
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
 FOREIGN KEY(affected_entity_id,project_id) REFERENCES entities(entity_id,project_id)
);
CREATE TABLE knowledge_records (
 knowledge_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL REFERENCES
projects(project_id),
 kind TEXT NOT NULL, title TEXT NOT NULL, source_language TEXT NOT NULL, current_revision_id
INTEGER,
 state TEXT NOT NULL CHECK(state IN
('verified','needs_review','inferred','rejected','retired')), owner TEXT NOT NULL,
 UNIQUE(knowledge_id,project_id),
 FOREIGN KEY(current_revision_id,knowledge_id) REFERENCES
knowledge_revisions(knowledge_revision_id,knowledge_id)
);
CREATE TABLE knowledge_revisions (
 knowledge_revision_id INTEGER PRIMARY KEY, knowledge_id INTEGER NOT NULL REFERENCES
knowledge_records(knowledge_id),
 body_md TEXT NOT NULL, editor TEXT NOT NULL, created_at TEXT NOT NULL,
 verification_state TEXT NOT NULL CHECK(verification_state IN
('verified','needs_review','inferred','rejected','retired')),
 verified_by TEXT, verified_at TEXT, UNIQUE(knowledge_revision_id,knowledge_id)
);
CREATE TABLE knowledge_links (
 link_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, knowledge_id INTEGER NOT NULL,
 entity_id INTEGER, relationship_id INTEGER, evidence_id INTEGER, link_role TEXT NOT NULL,
 last_verified_generation_id INTEGER,
 FOREIGN KEY(knowledge_id,project_id) REFERENCES knowledge_records(knowledge_id,project_id),
 FOREIGN KEY(entity_id,project_id) REFERENCES entities(entity_id,project_id),
 FOREIGN KEY(relationship_id,project_id) REFERENCES relationships(relationship_id,project_id),
 FOREIGN KEY(evidence_id,project_id) REFERENCES evidence(evidence_id,project_id),
 FOREIGN KEY(last_verified_generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
 CHECK(entity_id IS NOT NULL OR relationship_id IS NOT NULL OR evidence_id IS NOT NULL)
);
CREATE TABLE review_tasks (
 task_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, knowledge_id INTEGER, entity_id
INTEGER,
 reason TEXT NOT NULL, state TEXT NOT NULL CHECK(state IN ('pending','resolved')),
 opened_generation_id INTEGER, resolved_generation_id INTEGER, reviewer TEXT,
 FOREIGN KEY(project_id) REFERENCES projects(project_id),
```

```
 FOREIGN KEY(knowledge_id,project_id) REFERENCES knowledge_records(knowledge_id,project_id),
 FOREIGN KEY(entity_id,project_id) REFERENCES entities(entity_id,project_id),
 FOREIGN KEY(opened_generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
 FOREIGN KEY(resolved_generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id)
);
CREATE TABLE audit_events (
 event_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL REFERENCES projects(project_id),
 entity_type TEXT NOT NULL, entity_id INTEGER NOT NULL, action TEXT NOT NULL,
 actor TEXT NOT NULL, recorded_at TEXT NOT NULL, before_hash TEXT, after_hash TEXT
);
```

Indexes declared by this migration: idx_relationship_from, idx_relationship_to, idx_evidence_revision, idx_entity_file, idx_review_state, idx_knowledge_entity, idx_knowledge_record. Inspect their column/order clauses in the named migration when changing query behavior.

26.3. Migration 002 constraints

Additional relational and lifecycle protection beyond base table creation. Source: migrations/002_constraints.sql.

```
CREATE TABLE component_properties (
 component_entity_id INTEGER NOT NULL, generation_id INTEGER NOT NULL,
 property_name TEXT NOT NULL, serialized_value TEXT NOT NULL, value_kind TEXT NOT NULL,
evidence_id INTEGER NOT NULL,
 PRIMARY KEY(component_entity_id,generation_id,property_name),
 FOREIGN KEY(component_entity_id,generation_id) REFERENCES
entity_versions(entity_id,generation_id),
 FOREIGN KEY(evidence_id,generation_id) REFERENCES evidence(evidence_id,generation_id)
);
CREATE TABLE control_flow_nodes (
 cfg_node_id INTEGER PRIMARY KEY, routine_version_id INTEGER NOT NULL, generation_id INTEGER
NOT NULL,
 node_kind TEXT NOT NULL, source_range_json TEXT NOT NULL
CHECK(json_valid(source_range_json)), expression_ref TEXT,
 UNIQUE(cfg_node_id,generation_id),
 FOREIGN KEY(routine_version_id,generation_id) REFERENCES
entity_versions(entity_version_id,generation_id)
);
CREATE TABLE control_flow_edges (
 cfg_edge_id INTEGER PRIMARY KEY, generation_id INTEGER NOT NULL,
 from_cfg_node_id INTEGER NOT NULL, to_cfg_node_id INTEGER NOT NULL, edge_kind TEXT NOT NULL,
condition_ref TEXT,
 FOREIGN KEY(from_cfg_node_id,generation_id) REFERENCES
control_flow_nodes(cfg_node_id,generation_id),
 FOREIGN KEY(to_cfg_node_id,generation_id) REFERENCES
control_flow_nodes(cfg_node_id,generation_id)
);
CREATE TABLE translations (
 translation_id INTEGER PRIMARY KEY, knowledge_revision_id INTEGER NOT NULL REFERENCES
knowledge_revisions(knowledge_revision_id),
 target_locale TEXT NOT NULL, provider_id TEXT NOT NULL, provider_version TEXT NOT NULL,
translated_body TEXT NOT NULL,
```

```
 source_hash TEXT NOT NULL CHECK(length(source_hash)=64), translation_state TEXT NOT NULL,
reviewer TEXT,
 UNIQUE(knowledge_revision_id,target_locale,provider_id,provider_version)
);
CREATE TABLE artifacts (
 artifact_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, generation_id INTEGER NOT NULL,
 kind TEXT NOT NULL, artifact_path TEXT, blob_id TEXT REFERENCES artifact_blobs(blob_id),
 content_hash TEXT NOT NULL CHECK(length(content_hash)=64), locale TEXT NOT NULL,
template_version TEXT NOT NULL, build_state TEXT NOT NULL,
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id),
 CHECK((artifact_path IS NOT NULL AND blob_id IS NULL) OR (artifact_path IS NULL AND blob_id
IS NOT NULL))
);
CREATE TABLE export_jobs (
 export_id INTEGER PRIMARY KEY, project_id INTEGER NOT NULL, generation_id INTEGER NOT NULL,
 format TEXT NOT NULL, scope_json TEXT NOT NULL CHECK(json_valid(scope_json)), started_at TEXT
 NOT NULL,
 finished_at TEXT, manifest_hash TEXT, status TEXT NOT NULL,
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id)
);
```

Indexes declared by this migration: idx_entity_display, idx_artifacts_generation, idx_evidence_source_range. Inspect their column/order clauses in the named migration when changing query behavior.

Triggers declared by this migration: immutable_file_identity, immutable_entity_identity, revision_requires_capture, immutable_generation_update, immutable_generation_delete, valid_published_head, immutable_knowledge_update, immutable_knowledge_delete, immutable_revision_update, immutable_revision_delete, immutable_configuration_update, immutable_configuration_delete, captured_revision_matches, evidence_analyzer_matches, evidence_range_valid, generation_files_staging_insert, generation_files_immutable_update, generation_files_immutable_delete, entity_versions_staging_insert, entity_versions_immutable_update, entity_versions_immutable_delete, evidence_staging_insert, evidence_immutable_update, evidence_immutable_delete, relationships_staging_insert, relationships_immutable_update, relationships_immutable_delete, diagnostics_staging_insert, diagnostics_immutable_update, diagnostics_immutable_delete, component_properties_staging_insert, component_properties_immutable_update, component_properties_immutable_delete, control_flow_nodes_staging_insert, control_flow_nodes_immutable_update, control_flow_nodes_immutable_delete, control_flow_edges_staging_insert, control_flow_edges_immutable_update, control_flow_edges_immutable_delete. Their complete WHEN and RAISE/action bodies are authoritative in the migration; preserve them during migration and backup validation.

26.4. Migration 003 lifecycle

Managed cache, retention, release and continuity lifecycle support. Source: migrations/003_lifecycle.sql.

```
CREATE TABLE retention_policy (
 project_id INTEGER PRIMARY KEY REFERENCES projects(project_id),
 recent_generations INTEGER NOT NULL DEFAULT 10 CHECK(recent_generations BETWEEN 1 AND 10000)
);
CREATE TABLE retained_generations (
```

```
 project_id INTEGER NOT NULL REFERENCES projects(project_id),
 generation_id INTEGER NOT NULL,
 label TEXT NOT NULL, reviewer TEXT NOT NULL, recorded_at TEXT NOT NULL,
 PRIMARY KEY(project_id,generation_id),
 FOREIGN KEY(generation_id,project_id) REFERENCES
analysis_generations(generation_id,project_id)
);
```

26.5. Migration 004 search

Generation-scoped search indexes and their maintenance. Source: migrations/004_search.sql.

```
CREATE TABLE search_documents (
 document_id INTEGER PRIMARY KEY,
 generation_id INTEGER NOT NULL REFERENCES analysis_generations(generation_id),
 kind TEXT NOT NULL, title TEXT NOT NULL, body TEXT NOT NULL,
 entity_key TEXT NOT NULL DEFAULT '', knowledge_id INTEGER,
 source_path TEXT NOT NULL DEFAULT '', verification_state TEXT NOT NULL,
 FOREIGN KEY(knowledge_id) REFERENCES knowledge_records(knowledge_id)
);
CREATE TABLE search_terms (
 term TEXT COLLATE NOCASE NOT NULL,
 document_id INTEGER NOT NULL REFERENCES search_documents(document_id) ON DELETE CASCADE,
 PRIMARY KEY(term,document_id)
);
CREATE TABLE search_index_state (
 generation_id INTEGER PRIMARY KEY REFERENCES analysis_generations(generation_id),
 knowledge_revision INTEGER NOT NULL, search_version TEXT NOT NULL
);
```

Indexes declared by this migration: idx_search_generation. Inspect their column/order clauses in the named migration when changing query behavior.

26.6. Migration 005 document sections

Document section, audience and related knowledge organization. Source: migrations/005_document_sections.sql.

```
CREATE TABLE document_section_versions (
 revision_id INTEGER PRIMARY KEY REFERENCES knowledge_revisions(knowledge_revision_id),
 knowledge_id INTEGER NOT NULL REFERENCES knowledge_records(knowledge_id),
 section_id TEXT NOT NULL, document_kind TEXT NOT NULL, audience TEXT NOT NULL,
 source_generation INTEGER REFERENCES analysis_generations(generation_id),
 FOREIGN KEY(revision_id,knowledge_id) REFERENCES
knowledge_revisions(knowledge_revision_id,knowledge_id)
);
```

26.7. Migration 006 revision links

Exact revision-link targets and additional integrity constraints. Source: migrations/006_revision_links.sql.

```
CREATE TABLE knowledge_revision_links (
 revision_id INTEGER NOT NULL REFERENCES knowledge_revisions(knowledge_revision_id),
 ordinal INTEGER NOT NULL,
 target_kind TEXT NOT NULL CHECK(target_kind IN
```

```
('entity','relationship','evidence','screenshot','section')),
 target_id INTEGER NOT NULL, source_generation INTEGER REFERENCES
analysis_generations(generation_id),
 link_role TEXT NOT NULL CHECK(link_role IN ('explains','cites','depends_on','illustrates')),
 label TEXT NOT NULL,
 PRIMARY KEY(revision_id,ordinal), UNIQUE(revision_id,target_kind,target_id,link_role)
);
```

Indexes declared by this migration: knowledge_revision_links_target. Inspect their column/order clauses in the named migration when changing query behavior.

26.8. Migration 007 author questions

Attributed author-question metadata and answer provenance. Source: migrations/007_author_questions.sql.

```
CREATE TABLE knowledge_questions (
 knowledge_id INTEGER PRIMARY KEY REFERENCES knowledge_records(knowledge_id),
 question_key TEXT NOT NULL CHECK(length(question_key)=64),
 question_json TEXT NOT NULL CHECK(json_valid(question_json)),
 opened_generation INTEGER NOT NULL REFERENCES analysis_generations(generation_id)
);
```

Indexes declared by this migration: knowledge_questions_key. Inspect their column/order clauses in the named migration when changing query behavior.

26.9. Migration 008 translation cache

Revision/locale translation caching and named review support. Source: migrations/008_translation_cache.sql.

```
ALTER TABLE translations ADD COLUMN cache_key TEXT;
ALTER TABLE translations ADD COLUMN cache_json TEXT NOT NULL DEFAULT '{}'
CHECK(json_valid(cache_json));
```

Indexes declared by this migration: translations_cache_key. Inspect their column/order clauses in the named migration when changing query behavior.

26.10. Migration 009 structured knowledge

Typed authored metadata stored with its exact knowledge revision. Source: migrations/009_structured_knowledge.sql.

```
ALTER TABLE knowledge_revisions ADD COLUMN structured_json TEXT NOT NULL DEFAULT '{}'
CHECK(json_valid(structured_json) AND json_type(structured_json)='object');
```

27. Engineering change and release checklists

27.1. Before changing a contract

1. Identify the actual user learning problem and the layer that owns its behavior.

2. Inspect the relevant source, native form, persisted contract and focused tests.

3. Confirm repository state and take the required verified pre-change backup to the configured destination.

4. Define the expected before/after behavior and a meaningful validation example. Preserve original source fixtures and owner data.

5. Make the smallest approved change that satisfies the learning requirement without adding unrelated administration or helper layers.

27.2. Before declaring the change validated

1. Build the current application and run the serialized form baseline when applicable.
2. Run focused tests that exercise the changed contract and the necessary adjacent regressions.
3. Inspect native behavior at restored and maximized sizes when the UI changed.
4. Save/reopen typed knowledge and publish the same lesson when capture, projection or export changed.
5. Verify source bytes, authored history, current-source qualifications and privacy boundaries remain correct.
6. Update the matching manual sections and inspect their final rendered pages.

27.3. Before delivering a release

1. Build current application and validator executables and identify a fresh package version.
2. Package required files only, including current guides and help. Exclude developer-only lesson fixtures and owner data from the desktop delivery.
3. Verify extracted binary checksums and actual native startup. Exercise a disposable selected application's learner route in the extracted copy.
4. Build from a fresh source-ZIP extraction and retain its log.
5. Remove only confirmed disposable validation folders; preserve owner material and useful failure evidence.
6. Commit intended changes, push every configured repository and verify remote branch identities.